

MIM - Metamodel Informatiemodellering: Uitleg



Geonovum Standaard
Werkversie 31 juli 2026

Laatste werkversie:

<https://geonovum.github.io/NL-ReSpec-GN-template/>

Redacteur:

voornaam achternaam ([Geonovum](#))

Auteur:

voornaam achternaam ([Geonovum](#))

Doe mee:

[GitHub Geonovum/NL-ReSpec-GN-template](#)

[All issues](#)

[Dien een melding in](#)

[Revisiehistorie](#)

[Pull requests](#)

Dit document is ook beschikbaar in dit niet-normatieve formaat: [pdf](#)



Dit document valt onder de volgende licentie:

[Creative Commons Attribution 4.0 International Public License](#)

Samenvatting

Dit document is onderdeel van de documentatieset met betrekking tot het MIM metamodel.

Status van dit document

Dit is een werkversie die op elk moment kan worden gewijzigd, verwijderd of vervangen door andere documenten. Het is geen stabiel document.

Inhoudsopgave

Samenvatting

Status van dit document

1. Uitleg MIM Metamodel

1.1 Inleiding

1.1.1 Beschouwingsdomein en verwerkingsdomein

1.1.2 Soorten modellen

1.1.3 Notatiegebruik

1.2 Het MIM begrippenkader

1.2.1 De opzet van dit begrippenkader

1.2.2 De concrete zaken uit het beschouwingsdomein

1.2.2.1 De basis: domeinobjecten en kenmerken

1.2.2.2 Identificerend kenmerk en identificatoren

1.2.2.3 Kenmerken en invullingen

1.2.2.4 Eigenschappen en waarden

1.2.2.5 Classificaties en categoriën

1.2.2.6 Rollen en relaties

1.2.2.7 Relatie en rolinvullingen met kenmerken

1.2.2.8 Veranderlijkheid van kenmerken

1.2.3 De concrete zaken uit het verwerkingsdomein

1.2.3.1 Gegevens

1.2.3.2 Gegevensobject

1.2.3.3 Gebruik van identificerende kenmerken voor verwijzingen

1.2.3.4 Wat voorbeelden

1.2.4 Typering van domeinobjecten

1.2.4.1 Objecttype en populatie

1.2.4.2 Categoriseren of typeren: objecttype vs categorie

1.2.4.3 Feiten, gegevens, informatiebehoefte en kennis

1.2.4.4 Attribuuttypen en waarden

1.2.4.5 Relatietypen en rollen

1.2.4.6 Relatietype met kenmerken

1.2.4.7 Condities

1.2.4.8 Wat voorbeelden

1.2.4.9 Waardetypen vs categorieën en objecttypen

1.2.5 Typering van gegevens

1.2.5.1 Gegevenstypen en gegevensobjecttypen

1.2.5.2 Identificeren van het hoofdonderwerp: sleutels

1.2.5.3 Varianten van gegevenstypen

1.2.5.4	Gegevens over relaties
1.2.5.5	Speciaal soort gegevensobjecttypen
1.2.5.6	Speciaal soort gegevenstypen
1.2.5.7	Gegevensverzamelingen en referentielijsten
1.2.5.8	Gegevenstypen en gegevens m.b.t. waardelijsten
1.2.5.9	Gegevenstypen en gegevens m.b.t. classificatielijsten
1.2.5.10	Gegevenstypen en gegevens m.b.t. populaties

2. Conformiteit

A. Referenties

A.1 Normatieve referenties

§1. Uitleg MIM Metamodel

§1.1 Inleiding

Dit document beschrijft de begrippen die relevant zijn voor het MIM Metamodel. Gezamenlijk geven deze begrippen de terminologie, het vocabulaire, die voor MIM 2.0 wordt gehanteerd en de betekenis die daaraan moet worden toegekend binnen de context van MIM 2.0.

NB: Dit document bouwt het begrippenkader beetje-bij-beetje op, door steeds iets meer van het model prijs te geven. Het volledige begrippenkader is hier te vinden: [MIM-begrippenkader](#). Het volledige metamodel in grafische vorm is hier te vinden: [MIM-metamodel](#). Dit metamodel bestaat naast het begrippenkader ook uit een conceptueel (meta-)informatiemodel en een logisch (meta-)gegevensmodel.

Voor het beschrijven van de MIM-begrippen is de NL-SBB standaard gebruikt. Een belangrijk principe daarbij is het onderscheid tussen een «term», een «begrip» en de «definitie» van zo'n begrip:

- Een «term» is (slechts) een woord of aaneenschakeling van woorden. De term is hetgeen we in een bepaalde context gebruiken om te verwijzen naar een specifiek begrip. Zo wordt de term "bank" gebruikt om te verwijzen naar het begrip «bank (financieel)», maar ook naar het begrip «bank (meubelstuk)».
- Een «begrip» is de gedachte, de notie die we bedoelen als we een «term» gebruiken in een specifieke context. Als we (dus) dezelfde term gebruiken in twee verschillende contexten met hun eigen betekenis, dan hebben we het ook over twee verschillende begrippen. Andersom

kun je meerdere termen gebruiken voor hetzelfde begrip: de «voorkeursterm» of een «alternatieve term», indien hiervan sprake is.

- Een «definitie» is een gestructureerde tekstuele beschrijving van wat we bedoelen met een «begrip». Een «term» heeft (dus) nooit een definitie, maar een begrip wel. Een definitie is ook niet zomaar een stukje uitleg over de betekenis van een begrip. Een definitie volgt bepaalde regels, kent een specifieke structuur. Zo moet de definitie onderscheidend en precies genoeg zijn om begrippen van elkaar te kunnen onderscheiden, zodat je weet wat er wel en niet onder dat begrip wordt verstaan.

Het MIM metamodel heeft betrekking op zowel modellen van MIM niveau 2 als MIM niveau 3. Zie [soorten modellen](#) voor nadere uitleg hierover. Uitgangspunt bij dit begrippenmodel voor MIM is dat het op deze beide niveaus betrekking heeft: al deze begrippen horen tot hetzelfde begrippenkader. Conform de NL-SBB betekent dit dat we nooit dezelfde term gebruiken voor twee verschillende begrippen: we hanteren voor elk (meta)begrip in dit begrippenkader unieke termen.

Zo maken we (dus) ook onderscheid tussen «Objecttype» en «Gegevensobjecttype», omdat dit twee verschillende begrippen betreft. In de huidige versie van MIM kan hiervoor hetzelfde stereotype worden gebruikt: "objecttype". Uit de context is vervolgens duidelijk welk begrip wordt bedoeld: stereotype "objecttype" in een conceptueel informatiemodel (CIM) komt overeen met het begrip «Objecttype» en stereotype "objecttype" in een logisch gegevensmodel (LGM) komt overeen met het begrip «Gegevensobjecttype». Bij het lezen van dit document zal duidelijk worden dat we de term "Objecttype" **alleen** gebruiken voor het CIM, MIM niveau 2.

[!NOTE] Dit document gaat slechts over het begrippenkader. Het doet daarmee geen uitspraken of in een specifieke modelleertaal (UML, Linked Data, FBM, ER) de termen die voor deze begrippen worden gebruikt ook overgenomen moeten worden in deze modelleertalen. Hoewel dit bij bepaalde modelleertalen (zoals bij Linked Data of stereotypes in UML) voor de hand ligt, doet dit document hier geen uitspraken over.

§ 1.1.1 Beschouwingsdomein en verwerkingsdomein

Met gegevensmodelleren en gegevensduiding beschrijven we gegevens. Die gegevens *gaan* ergens over: bijvoorbeeld over personen, gebeurtenissen, objecten, plaatsen en de relaties daartussen. Mensen geven betekenis aan gegevens door zich een beeld te vormen waarover de gegevens gaan, maar bijvoorbeeld ook door beslissingen af te laten hangen van de gegevens die zij tot hun beschikking hebben of door in formele documenten afspraken op te schrijven welke betekenis we met bepaalde gegevens beogen. Om nauwkeurig uit te leggen wat gegevens betekenen, zullen we het over deze betekenis moeten hebben. En daarvoor is het van belang om bepaalde zaken van elkaar te onderscheiden en de relaties daartussen te benoemen.

Enerzijds kunnen we het hebben over het domein *waarover* we gegevens willen verwerken. En anderzijds kunnen we het over de gegevens (en de verwerking daarvan) *zelf* hebben. En dit zijn verschillende zaken! Eerstgenoemde domein zullen we het «*beschouwingsdomein*» noemen, laatstgenoemde het «*verwerkingsdomein*».

[!NOTE] Het «beschouwingsdomein» wordt in de Engelse literatuur ook wel "Universe of Discourse" genoemd. We hebben gekozen voor de term "beschouwingsdomein" om expliciet te maken dat het ons gaat om hetgeen we beschouwen. Dat kan de waarneembare werkelijkheid zijn, maar ook niet-waarneembare zaken zoals juridische aspecten of sociale afspraken of zelfs fictieve zaken (als dat hetgeen is dat je zou willen beschouwen). Daarnaast kun je in verschillende contexten anders tegen de werkelijkheid aankijken. Met de term "beschouwingsdomein" maken we duidelijk dat *hoe* je beschouwt ook van belang is. De term "verwerkingsdomein" is om vergelijkbare redenen gekozen. We kijken hier bewust naar het domein waarin de verwerking plaats vindt, niet het domein dat we beschouwen *bij* die verwerking. Ook is bewust gekozen om het woord "gegevens" niet op te nemen (waarmee je zou kunnen komen tot woorden als "gegevensdomein" of "gegevensverwerkingsdomein"). De reden daarvoor is dat we ons in het verwerkingsdomein op voorhand niet willen beperken tot gegevens. Ook regels en processen zou je kunnen beschrijven met betrekking tot het verwerkingsdomein.

We gebruiken hier steeds het begrip «verwerken». We hebben het dan ook over zowel het lezen, gebruiken, uitwisselen, creëren, opslaan, wijzigen, duurzaam bewaren als vernietigen van gegevens.

Aangezien gegevens over de dingen in het beschouwingsdomein gaan, zullen we die dingen moeten kunnen herkennen. Dit lijkt vaak makkelijk, maar dat is het niet. Zo is het relatief makkelijk om te bepalen of iets een mens is, of juist niet. Maar als het om meer abstracte zaken gaat (wanneer ben je Belastingplichtige, Verdachte of Eigenaar?), dan wordt het moeilijk. En als we dingen niet alleen willen onderscheiden (antwoord op de vraag: "is het er eentje? (en niet twee)"), of herkennen (antwoord op de vraag: "is het er zo eentje?"), maar ook identificeren (antwoord op de vraag: "is het die ene?"), dan wordt het nog een stuk moeilijker. Modelleren is een hulpmiddel om deze complexiteit behapbaar te maken.

§ 1.1.2 Soorten modellen

We modelleren zowel het beschouwingsdomein als het verwerkingsdomein. Met modelleren beschrijven we op een gestructureerde wijze een domein. En het is daarbij verstandig om niet alles in één model te willen stoppen: hierdoor wordt het model te complex en niet meer te begrijpen. Modelleren is juist een werkwijze om te komen tot een eenduidige representatie van het beschouwingsdomein dan wel verwerkingsdomein met precies voldoende detail voor optimaal begrip. Deze eenduidige werkwijze is afgestemd op het doel dat we willen bereiken met het model, wat we

ermee wensen te representeren. We onderscheiden vijf verschillende doelen, en evenzoveel soorten representaties:

0. **Tekstuele bronnen.** Voor mensen leesbare en begrijpbare vastgelegde beschrijvingen in natuurlijke taal van de kennis over het beschouwingsdomein. Uit deze bronnen kan de betekenis worden gevonden, in de zin van: "de betekenis is, zoals beoogd in dit document". Zo'n bron legitimeert de beoogde betekenis, bijvoorbeeld een wet of een standaard waar we ons aan willen houden. Maar ook tekst in de vorm van verhalen kunnen helpen: ze nemen de lezer mee in de betekenis vanuit voorbeelden en concrete gebeurtenissen.
1. **Model van begrippen.** Een model van begrippen helpt om een beter inzicht te krijgen in wat er wordt bedoeld als een bepaald woord of woordcombinatie ("term") wordt gebruikt in het beschouwingsdomein. We modelleren hier het begrip dat bestaat voor de communicatie in dat domein: welke woorden *daadwerkelijk* worden gebruikt in het beschouwingsdomein en wat ze in die context betekenen voor degenen die deze woorden gebruiken. Dergelijke woorden kunnen duiding geven aan een onderwerp van gesprek in de beschouwde werkelijkheid ("werkneemer", "woning") en ook aan afspraken of verplichtingen die in het beschouwingsdomein gelden ("arbeidscontract", "eigendom" of "belastingplicht").
2. **Conceptueel informatiemodel**, waarmee inzicht wordt gegeven welke dingen (zoals: objecten, actoren en handelingen) relevant zijn om te beschouwen en welke eigenschappen daarvan en relaties daartussen. Anders dan bij een model van begrippen, gaat het ons hier niet primair om de gebruikte taal, maar juist de dingen waarover wordt gesproken (letterlijk: "de onderwerpen van gesprek"). Het gaat ons om het *ontologisch commitment* dat we aangaan met betrekking tot het beschouwingsdomein. Welke dingen we in het domein willen onderscheiden, hoe we ze van elkaar kunnen onderscheiden en afzonderlijk kunnen identificeren. De afbakening van het model richt zich daarbij op datgeen we willen onderscheiden *omdat we er informatie over wensen*, omdat we het relevant vinden om er iets over de weten.
3. **Logisch gegevensmodel.** Waar we in de vorige modellen kijken naar het beschouwingsdomein waarover we gegevens willen verwerken, kijken we in dit model juist naar die gegevens zelf. Het logisch gegevensmodel is een model van het verwerkingsdomein. Het logisch gegevensmodel beschrijft de gegevens die in het verwerkingsdomein worden gebruikt. Het is een formele, complete specificatie van de gegevens die voor een bepaalde toepassing noodzakelijk zijn. Het is **geen** model van het beschouwingsdomein, hoewel de modellen veel op elkaar kunnen lijken (dit wordt isomorfie genoemd, hierover later meer). Bij het logisch gegevensmodel kunnen we een nader onderscheid maken tussen modellen van administraties (hoe gegevens worden opgeslagen), modellen van interacties (hoe gegevens worden uitgewisseld) en modellen van de verwerking binnen een proces (hoe gegevens worden gebruikt).
4. **Fysiek datamodel.** Tenslotte zullen gegevens ook daadwerkelijk gecreeërd en vastgelegd, uitgewisseld of bewerkt moeten worden. Het fysieke datamodel beschrijft hoe gegevens als data worden gecreeërd en vastgelegd, uitgewisseld of bewerkt in een specifiek technisch formaat.

Van modelsoort (1) naar modelsoort (3) wordt steeds duidelijker hoe we tegen de beschouwde werkelijkheid aankijken en daarbij alleen aspecten meenemen die we relevant vinden. Van modelsoort (3) naar (5) wordt duidelijk hoe de gegevensverwerking vorm krijgt inclusief de rationale achter die specifieke vorm. Daarbij is ook het fysiek datamodel nog slechts een model van de data: het betreft niet de data zelf, maar het beschrijft het "model", de "mal" waarbinnen de data moet passen. Daarnaast geldt dat er niet alleen een afhankelijkheid is van (1) naar (5), ook andersom is sprake van een afhankelijkheid: zo kun je het nooit over de onderwerpen in de beschouwde werkelijkheid hebben, als je daarover geen gegevens uitwisselt.

In dit document zullen we ons met name richten op het conceptueel informatiemodel en het logisch gegevensmodel. De samenhang met de overige soorten modellen valt voorlopig buiten de scope van dit document.

§ 1.1.3 Notatiegebruik

We introduceren in dit document bepaalde begrippen en gebruiken daar bepaalde termen voor. Daarbij is het soms van belang om expliciet te maken dat we een begrip bedoelen en soms expliciet van belang dat we een term bedoelen. De term "drie" heeft 4 letters, terwijl het begrip «drie» de betekenis heeft van het getal 3. Waar we verwijzen naar een term gebruiken we daarvoor aanhalingstekens (""), waar we verwijzen naar een begrip gebruiken we guillemets («»), zoals in het voorbeeld in deze paragraaf. In de lopende tekst zullen we ook definities voor dergelijke begrippen geven. Daar gebruiken we HOOFDLETTERS om aan te geven dat we een begrip bedoelen dat we ook definiëren in dit stuk. In voorbeelden verwijzen we vaak naar concrete dingen in het beschouwingsdomein. Als we het over [Jan] hebben, dan bedoelen we niet het woord "Jan" of het begrip «Jan», maar met [Jan] verwijzen we naar een concreet aanwijsbaar persoon die bekend is onder de naam "Jan". In dergelijke gevallen gebruiken we blokhaken ([]).

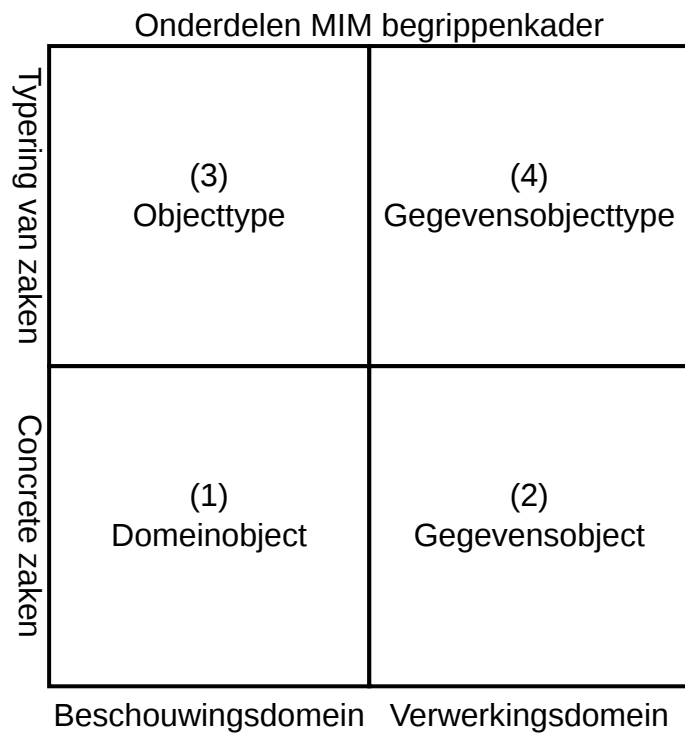
Ook hebben we het soms over een "Persoon" en bedoelen we het begrip «Persoon», terwijl we hiermee ook het gegevensobjecttype {Persoon} zouden kunnen bedoelen (gegevens over een persoon). Hiervoor maken we gebruik van accolades. We gebruiken ook accolades voor gegevenstypes, bijvoorbeeld {Persoon.naam} (het gegevenstype naam van een persoon).

§ 1.2 Het MIM begrippenkader

§ 1.2.1 De opzet van dit begrippenkader

Onderstaand figuur laat concreet zien hoe we dit begrippenkader opstellen. We onderscheiden vier onderdelen:

1. De concrete zaken uit het beschouwingsdomein. Hierbij gaat het om de begrippen die we gebruiken om de onderwerpen van gesprek te benoemen die concreet in het beschouwingsdomein bestaan. Een voorbeeld is het begrip «domeinobject»;
2. De concrete zaken uit het verwerkingsdomein. Hierbij gaat het om de begrippen die we gebruiken om de onderwerpen van gesprek te benoemen die concreet in het verwerkingsdomein bestaan. Een voorbeeld is het begrip «gegevensobject»;
3. De typering van zaken uit het beschouwingsdomein. Hierbij gaat het om de begrippen die we gebruiken om de elementen te benoemen die in modellen van het beschouwingsdomein te vinden zijn. Een voorbeeld is het begrip «objecttype»;
4. De typering van zaken uit het verwerkingsdomein. Hierbij gaat het om de begrippen die we gebruiken om de elementen te benoemen die in modellen van het verwerkingsdomein te vinden zijn. Een voorbeeld is het begrip «gegevensobjecttype».



§ 1.2.2 De concrete zaken uit het beschouwingsdomein

De termen "beschouwde werkelijkheid", "beschouwingsdomein" of kortweg "domein" kunnen in dit document als synoniem van elkaar worden gelezen met de volgende betekenis:

Een BESCHOUWINGSDOMEIN is een afgebakend deel van de werkelijkheid dat we relevant vinden om te beschouwen vanuit een bepaalde context

Belangrijke kenmerken van het beschouwingsdomein zijn:

- Het beperkt zich niet tot hetgeen we waar kunnen nemen, maar kan alles omvatten dat "we" als onderdeel van een werkelijkheid willen beschouwen;
- Het veronderstelt niet dat "de" werkelijkheid bestaat, maar dat je er vanuit een bepaalde invalshoek, een bepaalde context naar kijkt;
- Het beperkt zich tot het deel dat we relevant vinden om te beschouwen, dus zeker niet alles.

§ 1.2.2.1 De basis: domeinobjecten en kenmerken

Een DOMEINOBJECT is een onderscheidbaar en identificeerbaar iets in de beschouwde werkelijkheid

Wat we onderscheiden in een domein, en wat (dus) hier een domeinobject is, hangt af van wat we willen beschouwen. Deze keuzes over wat we willen onderscheiden leggen we vast in een conceptueel model. Een conceptueel model is, met andere woorden, een model van domeinobjecten. De term "domeinmodel" wordt dan ook wel gebruikt als synoniem voor een conceptueel model. Domeinobjecten kunnen zowel fysiek waarneembaar zijn, zoals een gebouw of voertuig, maar ook virtueel zoals giraal geld of het eigendom van een perceel. Ook hoeven domeinobjecten niet werkelijk te bestaan. Zo kun je het Star Wars domein beschouwen, en vanuit dat beschouwingsdomein gezien, bestaan zaken als light sabers en death stars.

Zo beschouwen zowel de BAG als de BRT bouwwerken, zoals het Station van Amersfoort. Echter, de BAG en de BRT beschouwen dit ene station op verschillende manieren. Er is hier dan ook sprake van twee domeinobjecten: het station Amersfoort, gezien vanuit het domein van de BAG en het station Amersfoort, gezien vanuit het domein van de BRT. Om dit ene station als hetzelfde domeinobject te beschouwen, is het noodzakelijk om op dezelfde manier naar dit station te kijken. De NEN3610 standaard is hier specifiek voor bedoeld. Deze standaard beschouwt een bouwwerk op een manier waarin zowel de betekenis van het BAG bouwwerk als het BRT bouwwerk vallen. Ook kan een betekenis uit een ander domein direct worden overgenomen. Zo wordt in verschillende registraties rechtstreeks gebruik gemaakt van het adres, zoals dit wordt beschouwd door de BAG. In dit geval wordt het "BAG-adres" dus op dezelfde wijze beschouwd en is ook sprake van hetzelfde domeinobject.

We gebruiken hier bewust het woord "domeinobject" om expliciet te maken dat we het over de objecten hebben die we beschouwen in een domein. Alles kan immers een object zijn, afhankelijk van wat je beschouwt. Zo hebben java-programmeurs het ook over objecten, maar dan bedoelen ze de java-objecten in hun programmeertaal (want dat is dan hetgeen ze beschouwen!). Verderop in dit document zullen we het hebben over gegevensobjecten. En dan bedoelen we dus ook weer die ob-

jecten die we beschouwen als we het over gegevens hebben, dwz: zoals ze voorkomen in het verwerkdomein.

[!NOTE] Stel dat we een conceptueel model zouden maken van de Java programmeertaal. Het beschouwingsdomein is dan (dus) de Java programmeertaal. En de domeinobjecten zijn dan (dus) onder meer de Java-objecten (en classes, interfaces, functions, etc). Op diezelfde manier kunnen we ook een conceptueel model maken van de gegevensverwerking. De domeinobjecten zijn dan (dus) onder meer de gegevensobjecten. Vaak worden dergelijke modellen ook wel *metamodellen* genoemd, omdat we de gegevensverwerking zelf beschouwen, dwz: een model-van-een-model, en gegevens-over-gegevens.

Een KENMERK is een verschijnsel dat toegekend kan worden aan bepaalde DOMEINOBJECTen

Als je een bepaald domeinobject beschrijft (bijvoorbeeld [Jan]), dan zul je het hebben over bepaalde kenmerken van Jan. Zoals het feit dat Jan een bepaalde lengte heeft, of getrouwd is, of ergens werkt. Dit zijn kenmerken van Jan, verschijnselen die we aan hem kunnen toekennen.

De definities van «domeinobject» en «kenmerk» zijn vrij ruim. Er kan van alles onder vallen. Dat is bewust: we willen immers zo'n beetje alles kunnen beschouwen wat we relevant vinden *om* te beschouwen. Toch is er uit de definities wel het één en ander te halen:

1. Een domeinobject is onderscheidbaar en identificeerbaar. Dit houdt in dat je over iets in het beschouwingsdomein kunt zeggen dat het er eentje is (en niet twee), dat het er zo-eentje is (en niet iets anders) en dat het die ene is (en niet een andere).
2. Een domeinobject is iets in de beschouwde werkelijkheid. Dit houdt in dat wat je onderscheid, bepaald wordt door het domein **en** hoe je dit domein wilt beschouwen. In verschillende domeinen of vanuit verschillende invalshoeken kun je dus ook andere dingen (willen) onderscheiden. We stellen hier ook dat DE werkelijkheid niet bestaat, maar dat je er altijd met een bepaalde blik naar kijkt: jouw invalshoek op de beschouwde werkelijkheid.
3. Een kenmerk is typisch voor bepaalde objecten in het domein. Dus bepaalde domeinobjecten *hebben* zo'n kenmerk. Zo'n kenmerk is overigens niet voorbehouden aan één type domeinobject. Zo kan het kenmerk «haarkleur» een kenmerk zijn van zowel (het haar van een voorkomen van) een mens, een konijn of zelfs een knuffelbeertje. We stellen hiermee alleen dat die objecten iets gemeen hebben: een haarkleur.
4. Wat een kenmerk is, is ook domeinspecifiek. In het vorige voorbeeld zou je ook kunnen stellen dat mensen, konijnen en knuffelbeertjes geen «haarkleur» hebben, maar dat «haarkleur» een kenmerk is van «haar», en dat een kenmerk van mensen, konijnen en knuffelbeertjes is dat ze «haar» kunnen hebben. Dat laatste is net wat preciezer, maar mogelijk niet relevant in jouw domein. Andersom kan het ook zijn dat we het juist (alleen) relevant vinden wie de kenmerk «rood haar» heeft. Als we dat domein beschouwen, dan kennen we bijvoorbeeld alleen het

kenmerk «rood haar», en kunnen we wel domeinobjecten onderscheiden in objecten die rood haar hebben, en de domeinobjecten die dat niet hebben. In dit domein kunnen we dan niet de domeinobjecten onderscheiden met een specifieke andere haarkleur.

Per type domeinobject kunnen er wel specifieke condities gelden voor een kenmerk. Zo zou het kunnen zijn dat de mogelijke haarkleuren per type domeinobject kan verschillen. Dit onderscheid komt niet tot uitdrukking in (de definitie van) een kenmerk, maar juist in de specificatie van de attributie van zo'n kenmerk aan een objecttype, zie hieronder bij de typering van domeinobjecten.

[!NOTE] Over een kenmerk van een domeinobject kun je uitspraken doen (ook wel *proposities* genoemd), zoals de uitspraak "De «haarkleur» van [Jan] is rood". In dit voorbeeld is [Jan] het domeinobject en «haarkleur» het kenmerk waarover we een uitspraak doen. Maar ook de volgende uitspraak is mogelijk: "[Jan] heeft «rood haar»". In dat laatste geval gaat het om het kenmerk «rood haar». Dit zijn twee verschillende manieren om het domein te beschouwen. Bij het eerste voorbeeld was het blijkbaar van belang om verschillende haarkleuren van elkaar te kunnen onderscheiden. Bij het tweede voorbeeld was alleen van belang OF er sprake was van rood haar.

§ 1.2.2.2 *Identificerend kenmerk en identificatoren*

Als we het over objecten in de fysieke werkelijkheid hebben, dan kunnen we die objecten aanwijzen. Zoals in de zin: "Hij daar, is de eigenaar". Stel dat die zin wordt uitgesproken in de winkel van Bakkerij Broodjes en er staan 5 mensen in die winkel, dan zul je de juiste persoon moeten aanwijzen! Je kunt ook gebruik maken van kenmerken die de juiste persoon identificeren, zoals in de zin: "Die lange man met rood haar daar, is Jan, de eigenaar". We willen het ook vaak kunnen hebben over domeinobjecten die we niet kunnen aanwijzen, maar wel kunnen identificeren. Bijvoorbeeld het huwelijk tussen [Jan] en [Marie]. Ook dan hebben we kenmerken van dat huwelijk nodig om het juiste huwelijk te kunnen identificeren.

Een IDENTIFICEREND KENMERK is een KENMERK waarmee de identiteit van een DOMEINOBJECT, zo nodig in combinatie met andere IDENTIFICERENDE KENMERKen, kan worden vastgesteld

Eén enkel identificerend kenmerk is vaak niet voldoende. Zo is in ons voorbeeld het niet voldoende om Jan daadwerkelijk uniek te kunnen identificeren: er zijn meer lange roodharige mannen op de wereld die Jan heten.

Een IDENTIFICATOR is een geheel van één of meerdere IDENTIFICERENDE KENMERKen waarmee de identiteit van een DOMEINOBJECT uniek kan worden vastgesteld

Een identifier zou bijvoorbeeld kunnen bestaan uit de geboortedatum, geboorteplaats, voornaam en achternaam van een persoon. Daarnaast is denkbaar dat een afzonderlijk kenmerk wordt toegekend aan een object, juist om deze uniek te identificeren. Zo'n kenmerk is vaak niet rechtstreeks met het object verbonden, maar wordt erbij gehouden. De enige manier om zo'n toekend kenmerk daadwerkelijk aan het domeinobject te verbinden is door het er letterlijk op te plakken. Bijvoorbeeld een serienummer dat gegraveerd wordt in het chassis van een voertuig of een straatnaambordje dat langs een weg wordt geplaatst.

Veel identificerende kenmerken zijn, goed beschouwd, vaak geen intrinsiek kenmerk van het object dat zij identificeren. Zo is een kenmerk van een motorvoertuig bijvoorbeeld dat het een motor, wielen, een kleur, een maximum snelheid, etc. heeft. Maar het kenteken van een motorvoertuig is geen echt kenmerk van dat motorvoertuig. Dit noemen we toegekende (identificerende) kenmerken. Het zijn kenmerken die we *beschouwen als kenmerk van* het object, terwijl ze feitelijk zijn toegekend. Het kenteken van een motorvoertuig is zo'n kenmerk.

Een TOEGEKEND IDENTIFICEREND KENMERK is een IDENTIFICEREND KENMERK dat is toegekend aan een DOMEINOBJECT om dit domeinobject uniek te kunnen identificeren

Om expliciet aan te geven dat een kenmerk daadwerkelijk **eigen** is aan een domeinobject, en dus niet is toegekend, kunnen we de term "intrinsiek kenmerk" gebruiken.

Merk op dat wat we intrinsiek vinden en wat we toegekend vinden vooral een domeinaangelegenheid is: het is maar net hoe je er naar kijkt. Dat is dan ook precies wat we in een conceptueel model aan het doen zijn. Zo kun je stellen dat de naam van een persoon een intrinsiek kenmerk is, maar feitelijk is ook die maar toegekend ("en we noemen haar..."). En ook van het geslacht van een persoon kun je stellen dat dit een intrinsiek kenmerk is, terwijl er ook beschouwingsdomeinen zijn waar dit eerder als een toegekend kenmerk wordt gezien. Van belang is dus meer *of* er in de beschouwde werkelijkheid sprake is van een kenmerk van een domeinobject, en niet zozeer of dit intrinsiek dan wel toegekend is.

§ 1.2.2.3 Kenmerken en invullingen

Als we naar de kenmerken van een domeinobject kijken, dan valt op dat er verschillende soorten kenmerken zijn te onderkennen. Een kenmerk van een domeinobject kent één of meerdere invullingen. We leggen dit uit aan de hand van vier voorbeeld-zinnen:

1. [Jan] heeft «lengte» "2 meter en 5 centimeter";
2. [Jan] heeft «geslacht» «man»;
3. [Jan] heeft «werkgever» [Bakkerij Broodjes];

4. [Jan] heeft «beroep» «Bakker».

De eerste en derde zin betreffen kenmerken van Jan waarvan de invulling *concreet* is. In de eerste zin gaat het om een letterlijke waarde, en in de derde zin gaat het om een ander domeinobject. De tweede en vierde zin betreffen kenmerken van Jan waarvan de invulling *conceptueel* is. Er wordt verwezen naar een bepaald begrip dat in dit beschouwingsdomein bestaat, het verwijst naar een achterliggende betekenis, de betekenis van wat een «man» is of wat een «Bakker» is.

De eerste en tweede zin betreffen *inherente* kenmerken van Jan. De kenmerken kunnen mogelijk van waarde veranderen, maar dat hangt volledig af van het domeinobject Jan zelf. De derde en vierde zin betreffen *relationele* kenmerken van Jan. De kenmerken zijn afhankelijk van de relatie die Jan heeft met zijn omgeving. In de derde zin gaat het om een relatie met een ander domeinobject, in de vierde zin gaat het om een relatie met de uitoefening van een beroep.

Soorten kenmerken					
Concreet	<table border="1"> <tr> <td>Eigenschap (<i>lengte, leeftijd</i>)</td><td>Rol (<i>werkgever, partner</i>)</td></tr> <tr> <td>Categorisch kenmerk (<i>geslacht, bio. soort</i>)</td><td>Categorisch kenmerk (<i>beroep, functie</i>)</td></tr> </table>	Eigenschap (<i>lengte, leeftijd</i>)	Rol (<i>werkgever, partner</i>)	Categorisch kenmerk (<i>geslacht, bio. soort</i>)	Categorisch kenmerk (<i>beroep, functie</i>)
Eigenschap (<i>lengte, leeftijd</i>)	Rol (<i>werkgever, partner</i>)				
Categorisch kenmerk (<i>geslacht, bio. soort</i>)	Categorisch kenmerk (<i>beroep, functie</i>)				
Conceptueel					
	Inherent Relationeel				

Eigenschap (<i>lengte, leeftijd</i>)	Rol (<i>werkgever, partner</i>)
Categorisch kenmerk (<i>geslacht, bio. soort</i>)	Categorisch kenmerk (<i>beroep, functie</i>)

Met deze twee onderscheidene elementen (concreet/conceptueel, inherent/relatieel) kunnen we vier soorten kenmerken onderscheiden. De invulling van deze vier soorten kenmerken verschilt, zoals ook zichtbaar is in de voorbeelden. De nauwlettende lezer ziet dat we bij de invullingen steeds andere leestekens gebruiken!

- Inherent concrete kenmerken noemen we *eigenschappen* en hebben als invulling een letterlijke waarde;
- Relationele concrete kenmerken noemen we *rollen* en hebben als invulling een domeinobject;

- Conceptuele kenmerken (zowel inherent als relationeel) noemen we *categorische kenmerken* en hebben als invulling een categorie.

[!NOTE] Merk op dat we met deze definitie van kenmerk nog niets zeggen over het aantal keer dat een kenmerk kan worden ingevuld voor een domeinobject. Zo kan [Jan] wel drie voornamen hebben, dus er zijn dan drie invullingen van het kenmerk «naam». Dergelijke beperkingen op het aantal keren dat een kenmerk ingevuld mag worden, zijn onderdeel van de condities die beschreven zijn in het model, in dit geval de cardinaliteit. Zie hiervoor de sectie over condities hieronder.

In de volgende secties werken we deze verschillende soorten kenmerken verder uit.

§ 1.2.2.4 Eigenschappen en waarden

Een EIGENSCHAP is een KENMERK van een DOMEINOBJECT waar uitsluitend een WAARDE aan kan worden toegekend

Een LETTERLIJKE WAARDE is een WAARDE waarvan de betekenis letterlijk genomen moet worden, de waarde zelf en niets meer

Een eigenschap is een kenmerk waar "slechts" een (letterlijke) waarde aan kan worden toegekend. Zoals «lengte» in het voorgaande voorbeeld. Een letterlijke waarde is geen domeinobject of verwijzing daarnaar, maar juist een letterlijke invulling van het kenmerk. De betekenis van de letterlijke waarde is niets anders dan de letterlijke waarde zelf. Zoals een getal, een stukje tekst, een datum of een boolean (waar/onwaar). Zo is in ons voorbeeld «(heeft) leeftijd» een kenmerk van [Jan]. De *invulling* van dit kenmerk is een waarde, bijvoorbeeld het getal 25. Daarmee is dit kenmerk een eigenschap van Jan. En ook het kenmerk «(heeft) naam» van [Jan] is een eigenschap, met de waarde "Jan" (een woord bestaande uit drie letters).

In de voorbeelden hebben we gezien dat een waarde vaak toch net iets meer is dan alleen een letterlijke waarde. De waarde "25 euro" bestaat feitelijk uit een letterlijke waarde (het getal 25) en een waarde die refereert aan een categorie (de valutacategorie «euro»). Een dergelijke waarde noemen we een complexe waarde.

Een COMPLEXE WAARDE is een WAARDE die bestaat uit een samenstel van afzonderlijk benoemde LETTERLIJKE WAARDEN en/of CATEGORIEën

§ 1.2.2.5 Classificaties en categoriën

Een CATEGORISCH KENMERK is een KENMERK van een DOMEINOBJECT waar een CATEGORIE aan kan worden toegekend

Een CATEGORIE is een aanduiding van een groep DOMEINOBJECTen die een kwaliteit gemeen hebben

Voorbeelden van categorieën zijn: kleur, geslacht, type voertuig (auto, fiets, bus) of merk. Je kunt nog onderscheid maken tussen nominale categorische kenmerken (de categorieën hebben geen volgorde) en ordinale categorische kenmerken (er is een volgorde, bijvoorbeeld urgentie: laag, midden, hoog).

De invulling van een categorisch kenmerk is geen letterlijke waarde, maar een verwijzing naar iets anders: een categorie. En deze categorie kan zelf ook weer eigenschappen hebben of relaties hebben. Zo kan bijvoorbeeld van de valutacategorie «euro» vastgelegd worden in welke landen een «euro» voorkomt. En net als bij een (letterlijk) kenmerk behoort een categorisch kenmerk bij één domeinobject en is deze niet afhankelijk van een relatie met een ander domeinobject.

Een categorie is een aanduiding voor een groep van objecten die iets gemeen hebben. Een categorisch kenmerk verbindt een domeinobject met die categorie. Zo kun je bijvoorbeeld objecten groeperen die levende wezens zijn en waarvan de moeder haar jongen melk geeft, dwz: de groep van zoogdieren. De categorie «zoogdier» kan dan de invulling zijn van het kenmerk «biologische soort» van een levend wezen.

Categorieën zijn vaak begrippen die al zijn onderkend in het begrippenmodel. Ook lijken categorieën veel op de typering van domeinobjecten die verderop in dit document aan bod komt. Dat is geen toeval: ook bij het typeren van domeinobjecten kijk je naar overeenkomstige kenmerken. Het verschil tussen categorieën en objecttypen wordt vooral bepaald door de manier waarop je een domein wenst te beschouwen. Zie hiervoor de sectie "categoriseren of typeren".

§ 1.2.2.6 Rollen en relaties

Een ROL is een KENMERK dat wordt vervuld door een DOMEINOBJECT in een RELATIE

Een RELATIE is een verbintenis tussen DOMEINOBJECTen

Een rol bestaat altijd in de context van een relatie. Dat is anders dan bij een eigenschap. Voor een eigenschap is alleen maar nodig dat je weet over welk domeinobject het gaat, maar bij een rol heb je ook de relatie nodig. Een relatie lijkt op een domeinobject. Net als bij een domeinobject kan een relatie zelf ook kenmerken hebben. Het verschil met een domeinobject is dat een relatie altijd afhankelijk is in zijn bestaan van andere domeinobjecten. De arbeidsrelatie tussen Jan en Bakkerij Broodjes is een voorbeeld van een relatie. In deze specifieke relatie ligt de *invulling* van de rol werkgever bij [Bakkerij Broodjes] en de *invulling* van de rol werknemer bij [Jan]. Zonder deze twee invullingen, kan er ook geen sprake zijn van de betreffende relatie.

Rollen zijn primair kenmerkend aan een relatie. Zo zijn de rollen «werkgever» en «werknemer» kenmerken die slechts betekenis hebben in de context van een arbeidsrelatie. Zo kun je de volgende uitspraken doen:

- De [Arbeidsrelatie tussen Jan en Bakkerij Broodjes] betreft «werknemer» [Jan];
- De [Arbeidsrelatie tussen Jan en Bakkerij Broodjes] betreft «werkgever» [Bakkerij Broodjes];
- De [Arbeidsrelatie tussen Jan en Bakkerij Broodjes] heeft «begindatum» "20 februari 2025".

Relaties hebben in beginsel geen richting: de arbeidsrelatie is *tussen* [Jan] en [Bakkerij Broodjes]. Als we het over deze relatie gaan hebben, dan gebruiken we vaak *proposities*: uitspraken die iets zeggen over de relatie, bijvoorbeeld:

- [Jan] *is in dienst van* [Bakkerij Broodjes];
- [Bakkerij Broodjes] *heeft werknemer* [Jan];
- [Jan] *heeft werkgever* [Bakkerij Broodjes].

Meerdere proposities zijn mogelijk bij één relatie. Soms wordt in zo'n propositie gebruik gemaakt van een rol. Dit is te zien in het 2e en 3e voorbeeld. De rol wordt daarmee een relationeel kenmerk van het domeinobject. Net zo goed als je kunt vragen "Wat is de leeftijd van Jan" (een vraag over de eigenschap «leeftijd» van een persoon), kun je de vraag stellen "Wie is de werkgever van Jan". De rol van werkgever wordt hier bijvoorbeeld vervuld door [Bakkerij Broodjes]. En [Jan] zelf vervult de rol van werknemer. In dit geval is «(heeft) werkgever» dus een relationeel kenmerk van [Jan]. De *invulling* van die rol ligt bij [Bakkerij Broodjes].

We bedoelen hier dus echt [Bakkerij Broodjes] *zelf*. Dit is wezenlijk anders dan de eigenschap «naam» van die bakkerij. Als we zouden zeggen: "De werkgever van Jan is 'Bakkerij broodjes'", dan lijkt het alsof de werkgever van [Jan] een naam is die bestaat uit twee woorden (!). in bovenstaande zin wordt met de aanduiding [Bakkerij Broodjes] (dus) verwezen naar de Bakkerij zelf en niet naar de naam van de Bakkerij.

[!NOTE] Om bovenstaand verschil nog beter te doorgronden, kun je de vraag stellen: "Hoe lang bestaat Bakkerij Broodjes al?". Stel dat [Bakkerij Broodjes] is begonnen met broodjes bakken in 1925. Dan is het antwoord op deze vraag (dus) 100 jaar in 2025. Maar stel dat Bakkerij Broodjes van naam is veranderd in 2020 en oorspronkelijk "Bakkerij op de hoek" genoemd werd. Ook dan blijft het antwoord op bovenstaande vraag hetzelfde. Wel zou een andere vraag zijn: "Hoe lang heet Bakkerij Broodjes al zo?". Dan is het antwoord: pas 5 jaar.

§ 1.2.2.7 Relatie en rolinvullingen met kenmerken

In onze kijk op het beschouwingsdomein hebben we het over domeinobjecten gehad die kenmerken kunnen hebben en waarbij sprake kan zijn van relaties tussen domeinobjecten. Ook hebben we laten zien dat relaties kenmerken kunnen hebben, zonder dat we de relatie zelf als een (afzonderlijk) domeinobject zien. We vonden de relatie niet "bijzonder" genoeg om deze daadwerkelijk te onderscheiden als afzonderlijk domeinobject. Uiteindelijk is dit echter een keuze hoe je het domein wilt beschouwen.

Een verbintenis tussen twee domeinobjecten kan zo belangrijk zijn in een domein, dat we zo'n verbintenis beschouwen als een afzonderlijk domeinobject en niet alleen als een relatie. Zo kunnen we het bijvoorbeeld over het huwelijk tussen Jan en Marie hebben. Dit domeinobject kan zelf kenmerken hebben, zoals de huwelijksdatum, de gehuwden en getuigen. En we zullen zo'n domeinobject moeten kunnen identificeren. In dit geval zou de combinatie van de identificerende kenmerken van de huwelijkspartners en de huwelijksdatum daarvoor bruikbaar zijn. Het is echter ook denkbaar dat in het betreffende domein een afzonderlijk huwelijkskenmerk bestaat, bijvoorbeeld het nummer op de huwelijksakte. In dit voorbeeld beschouwen we dit huwelijks-fenomeen (dus) niet als een relatie tussen Jan en Marie, maar als "een huwelijk": een afzonderlijk domeinobject.

Relaties hebben vaak te maken met gebeurtenissen van waaruit de relatie ontstaat. In bovenstaand voorbeeld stond de gebeurtenis centraal. Maar ook het resultaat van de gebeurtenis kan gezien worden als domeinobject. In zo'n geval zou huwelijk mogelijk een kenmerk «huwelijkse voorwaarden» hebben.

Bij het betalen van een rekening is zo sprake van de gebeurtenis van betalen, het resultaat van de betaling en de rollen betaler en ontvanger. Het is aan de modelleur om aan te geven welke onderdelen hij hiervan als domeinobjecten en kenmerken wenst te zien in het beschouwingsdomein.

En ook de rol(invulling) zelf kunnen we zien als een domeinobject. Zo is een kenmerk van een betaling de rol «betaler». Als we naar de invulling van die rol kijken, dan wordt die ingevuld door een domeinobject dat we de «Betaler» kunnen noemen. Met andere woorden:

Als Bakkerij Broodjes het loon van Jan overmaakt (4700 euro), dan:

- Is er sprake van een betaling;
- Een kenmerk van die betaling is het «bedrag»;
- De invulling van dit kenmerk voor deze betaling is "4700 euro";
- Een kenmerk van die betaling is de «betaler»;
- De invulling van dit kenmerk voor deze betaling is [Bakkerij Broodjes]
- [Bakkerij Broodjes] is in deze betaling de betaler.

[!NOTE] MIM is een metamodel. Het is geen aanpak om te komen tot het juiste model. MIM geeft dan ook geen specifieke richtlijnen of je een relatie c.q. rolinvulling als domeinobject zou moeten zien. Van belang is dat MIM geen onderscheid maakt tussen domeinobjecten die daadwerkelijk waarneembaar zijn en domeinobjecten die mogelijk ook als een relatie of rolinvulling kunnen worden gezien. Dit volgt uit de definitie van een domeinobject (zie ook de uitleg bij de definitie van het begrip «domeinobject» hierboven: "domeinobjecten kunnen zowel fysiek waarneembaar als virtueel zijn"). Ook maakt dit het makkelijker om MIM als uitwisselingsstandaard te gebruiken: een meer uitgebreide vorm van MIM zou het lastig maken om MIM uit te drukken in modelleertalen die dergelijke uitbreidingen niet kennen.

§ 1.2.2.8 Veranderlijkheid van kenmerken

Er zijn kenmerken van domeinobjecten die onveranderlijk zijn. Zo zal het kenmerk «geboortedatum» van een persoon nooit veranderen: een persoon is nu eenmaal geboren op een bepaalde dag, dat zal niet meer veranderen. Het kenmerk «leeftijd» verandert daarentegen elk jaar, maar wel op een hele reguliere manier (elk jaar eentje erbij). Er zijn ook kenmerken die veranderen, zonder dat sprake is van vooraf duidelijke manier. Zo zijn rollen over het algemeen veranderlijk: [Jan] is (nu) werknemer van [Bakkerij Broodjes], maar mogelijk is hij dat over een tijdje niet meer, en er is ook een tijd geweest dat hij het (nog) niet was.

Zo zijn er ook categorische kenmerken die onveranderlijk zijn en categorische kenmerken die juist veranderlijk zijn. Zo zal een levend wezen altijd tot de categorie van zoogdieren behoren, of juist niet. Het kenmerk «(is) werkloos» is echt juist weer veranderlijk, en hangt bovendien ook samen met de rol «werknemer».

[!NOTE] Je zult wellicht vinden dat een kenmerk «werkloos» niet gemodelleerd zou moeten worden in een conceptueel model. Vanuit de overtuiging dat de arbeidsrelatie de werkelijk juiste modellering is. Bedenk dan dat we een beschouwingsdomein aan het modelleren zijn. Wat we willen beschouwen, hangt van het domein af. Dus ook of we het interessant vinden om alle details te weten over de reden waarom iemand werkloos is. Dat kan nuttig zijn, maar ook volledig overbodig. Daarom kan een dergelijk kenmerk Waarde hebben om te modelleren. Een herkenbaar voorbeeld is dat je minimaal 18 jaar moet zijn om alcohol te mogen kopen. In dit beschouwingsdomein is de geboortedatum van een koper in het geheel niet relevant (en zelfs niet zijn of haar exacte leeftijd), slechts het kenmerk «is minimaal 18 jaar» is relevant in dit domein.

§ 1.2.3 De concrete zaken uit het verwerkingsdomein

In het vorige hoofdstuk hebben we de concrete zaken uit het beschouwingsdomein besproken. In dit hoofdstuk gaan we verder met de concrete zaken uit het verwerkingsdomein.

Waar we het over "verwerkingsdomein" hebben, bedoelen we expliciet het domein van de gegevensverwerking, dat je ook het "gegevensdomein" of het "gegevensverwerkingsdomein" zou kunnen noemen met de volgende betekenis:

Een VERWERKINGSDOMEIN is een afgebakend deel van de werkelijkheid waarin gegevens worden verwerkt over het BESCHOUWINGSDOMEIN

Belangrijke kenmerken van het verwerkingsdomein zijn:

- Ook het verwerkingsdomein is een deel de werkelijkheid: de administratieve werkelijkheid. Immers: het vastleggen van gegevens leidt daadwerkelijk tot iets. Bij fysieke vastlegging op papier zelfs tastbaar, maar ook bij digitale vastlegging is er sprake van verandering in deze (digitale) werkelijkheid.
- Het gaat ons hier om de verwerking van gegevens, waarbij we die gegevens zelf zien als de te verwerken eenheden;
- Het verwerkingsdomein is altijd verbonden aan één of meerdere beschouwingsdomeinen waarover de gegevens gaan die we verwerken, het kan daar niet los van worden gezien.

§ 1.2.3.1 Gegevens

Een GEGEVEN is een vastgelegde uitdrukking over hetzij een KENMERK van een DOMEIN-OBJECT dan wel een RELATIE tussen DOMEINOBJECTen

Kenmerkend aan een gegeven zijn de volgende elementen:

1. Een gegeven is **vastgelegd**, op wat voor manier dan ook: van een kleitablet, papieren document tot een elektronisch bericht. En van een papieren register (boekhouding, teboekstelling) tot een digitale database. In elk van deze vormen van sprake zijn van meer of minder mate van vluchtigheid of persistentie: gegevens in een bericht zullen minder persistent zijn dan gegevens die duurzaam worden opgeslagen in een administratie.
2. Een gegeven is een **uitdrukking**: zo'n uitdrukking kan bijvoorbeeld een bewering, stelling of waarneming zijn en zo'n uitdrukking kan gedaan worden door een mens of een sensor. Ook kan zo'n uitdrukking betrekking hebben op een meting, berekening of afleiding. Een uitdrukking hoeft ook niet waar te zijn, maar ze zijn in ieder geval *gedaan* door iets (bv een sensor, of een algoritme) of iemand (een daadwerkelijke persoon). Gegevens kunnen vastleggingen zijn van directe waarnemingen (een sensor die iets meet of een persoon die iets ziet), maar ook beweringen zijn op grond van afleidingen en beoordelingen (een algoritme dat een berekening uitvoert of een persoon die iets concludeert op basis van andere gegevens).
3. Een gegeven gaat over een **domeinobject**: het gaat over iets: de domeinobjecten die relevant zijn in het beschouwingsdomein waarover we gegevens verwerken.
4. Een gegeven betreft een **kenmerk** van dat domeinobject of **relatie** tussen domeinobjecten: het gegeven legt een uitdrukking vast over een kenmerk die het domeinobject heeft dan wel over een relatie tussen domeinobjecten. Bijvoorbeeld het kenmerk «geboortedatum» van de persoon [Jan] uit ons voorbeeld. Kenmerken hebben een naam (zoals "geboortedatum") en zijn getypeerd (het is helder wat we ermee bedoelen en welke invullingen mogelijk zijn, zoals een datum bij een geboortedatum).

[!NOTE] Een gegeven over een relatie tussen domeinobjecten kan op verschillende manieren worden uitgedrukt. Zo kan de arbeidsrelatie tussen [Jan] en [Bakkerij Broodjes] uitgedrukt worden met een gegeven waarin de relatie centraal staat: "Er is een dienstbetrekking tussen «werknemer» [Jan] en «werkgever» [Bakkerij Broodjes]", maar ook een gegeven waarbij één van de twee domeinobjecten centraal staat: "[Jan] (heeft) «werkgever» [Bakkerij Broodjes]". Het is aan de modelleur van het logisch gegevensmodel om hierin een keuze te maken. Zie ook verderop bij Relatietype en rollen.

Hoewel gegevens gaan over de domeinobjecten, hun kenmerken en relaties, is het niet direct mogelijk om de gegevens hiermee te verbinden. Gegevens bestaan, zo gezegd, eigenlijk altijd in een andere werkelijkheid dan domeinobjecten. Denk bijvoorbeeld aan een domeinobject als een persoon, of een voertuig of een weg. Van alle drie kun je gegevens vastleggen. Maar om daarbij de relatie te leggen *waarover* deze gegevens gaan, zitten we met een probleem. De enige manier om gegevens en domeinobjecten direct aan elkaar te relateren is letterlijk de gegevens op het domeinobject te "plakken", of (net zo letterlijk): het domeinobject te "oormerken" (!). In onze huidige, digitale, samenleving is dit een uitzondering. We hebben "in" de werkelijkheid van de gegevens iets nodig om te kunnen verwijzen naar de betreffende persoon, voertuig of weg. We moeten ze identificeren. Vaak gebruiken we een toegekend identificerend kenmerk om bij een gegeven aan te kunnen geven over welk domeinobject het gaat.

In ons voorbeeld kunnen we bijvoorbeeld het volgende gegeven uitdrukken, door gebruik te maken van de toegekende identificerend kenmerk «BSN»: "De persoon met BSN 123456782 heeft geboortedatum 10 februari 1970". Dit gegeven is een bewering over de eigenschap «geboortedatum» van het domeinobject dat geïdentificeerd kan worden met het toegekend identificerend kenmerk «BSN» met de waarde "123456782", dwz: onze [Jan].

NB: Van belang in bovenstaande definitie is dat een gegeven afhankelijk is van de domeinobjecten en kenmerken die je wilt onderkennen! Als je bijvoorbeeld een gegeven over de geboortedatum van Jan wilt vastleggen, dan veronderstelt dit dat je domeinobjecten zoals Jan wilt onderkennen, en daarvan het kenmerk geboortedatum. Hoewel je technisch gezien de uitspraak "De geboortedatum van Jan is 10 februari 1970" kunt vastleggen als gegeven, is een correcte interpretatie van deze uitspraak moeilijk zonder inzicht in het domein, en daarmee welke domeinobjecten en kenmerken je wilt onderkennen.

Een GEGEVENSGROEP is een groepering van GEGEVENS

Een gegevensgroep is simpelweg het groeperen van enkele gegevens die we op een bepaalde manier bij elkaar vinden horen. Dus bijvoorbeeld een lijstje van geboortedata van personen (we groeperen dan op die eigenschap), of de naam, adres en woonplaats gegevens over hetzelfde domeinobject, bijvoorbeeld onze [Jan]. Merk op dat een gegevensgroep (dus) ook het resultaat kan zijn van een query op een database die via een bericht wordt gedeeld. Het begrip «gegevensgroep» is de basisbouwsteen, bij het modelleren zal het veel vaker gaan over een speciaal soort gegevensgroep: een gegevensobject.

§ 1.2.3.2 Gegevensobject

Een GEGEVENSOBJECT is een onderscheidbaar geheel van GEGEVENS die als kleinste eenheid wordt verwerkt

Bij de verwerking van gegevens zal sprake zijn van een geheel van gegevens die als eenheid wordt verwerkt. Dit zou precies één gegeven kunnen zijn, maar vaak is sprake van de verwerking van enkele gegevens samen, bijvoorbeeld alle persoonsgegevens van [Jan]. We hebben het dan over een gegevensobject. Zo'n object kan gegevens omvatten die gaan over precies één domeinobject (het hoofdonderwerp van het gegevensobject), maar ook over meerdere domeinobjecten die aan elkaar gerelateerd zijn. De definitie van gegevensobject lijkt sterk op die van een domeinobject. Ook hier gaat het om een onderscheidbaar 'iets' in een domein, namelijk: het domein van de gegevensverwerking. En dat 'iets' is altijd een geheel van gegevens. Elk afzonderlijk gegeven gaat vervolgens over een domeinobject, waarmee het gegevensobject gaat over al deze domeinobjecten gezamenlijk. Als alle gegevens over hetzelfde domeinobject gaan, dan gaat ook het gegevensobject over precies dit ene domeinobject. Als een gegevensobject vooral gaat over één domeinobject, dan noemen we dat domeinobject het *hoofdonderwerp* van het gegevensobject.

Een HOOFDONDERWERP is een DOMEINOBJECT waarover een GEGEVENSOBJECT in hoofdzaak gaat

Merk op: hoewel de identiteit van een gegevensobject gerelateerd is aan de identiteit van het hoofdonderwerp, verschillen ze van elkaar. Zo kun je twee gegevensobjecten hebben die gaan over hetzelfde domeinobject, bijvoorbeeld één gegevensobject met de medische gegevens en één gegevensobject met (alleen) de naam-adres-woonplaats gegevens. Of één gegevensobject met de gegevens zoals deze op in 2015 geldig waren, en een ander gegevensobject met de gegevens zoals deze op 2016 geldig waren.

[!NOTE] Een gegevensobject kun je *zelf* ook weer zien als een domeinobject. Dus het is niet zo maar een geheel van gegevens, het is een geheel van gegevens die onderscheidbaar zijn in een domein. En in dit geval is dit het verwerkingsdomein van gegevens. Meta-gegevens (gegevens *over* gegevens) kunnen zo ook worden uitgedrukt: een (meta)gegeven is daarmee een vastgelegde uitdrukking over een getypeerde eigenschap van een gegevensobject. Merk op dat als we het hebben over gegevens-over-gegevens, we het over metagegevens hebben. Dergelijke meta-gegevens bestaan in dit geval *wel* in dezelfde werkelijkheid als de gegevens waarover ze gaan. Het is dan over het algemeen ook veel makkelijker om metagegevens en gegevens bij elkaar te plaatsen. Wel blijft van belang om te blijven erkennen dat het hier om twee verschillende objecten gaat. Onderstaand voorbeeld geeft dit weer:

- Een gegeven: "De persoon met BSN 123456782 heeft geboortedatum 10 februari 1970"
- Een metagegeven: "Het gegeven: 'De persoon met BSN 123456782 heeft geboortedatum 10 februari 1970' bevat een foutieve waarde voor de eigenschap BSN" Het gegeven *gaat over* een persoon, het metagegeven *gaat over* een gegeven.

§ 1.2.3.3 Gebruik van identificerende kenmerken voor verwijzingen

Een gegeven gaat over een kenmerk van een domeinobject of relatie tussen domeinobjecten. Van belang is dan ook om naar zo'n domeinobject te kunnen verwijzen. Hiervoor kunnen we de identifier (en daarmee de identificerende kenmerken van een domeinobject) gebruiken. De invulling van de identificerende kenmerken die we gebruiken om te verwijzen naar een specifiek domeinobject noemen we een *sleutelwaarde*

Een SLEUTELWAARDE is de invulling van één of meerdere KENMERKEN die gezamenlijk één enkel DOMEINOBJECT uniek aanduiden

Zo is de waarde "123456782" van een BSN een unieke sleutelwaarde voor een specifieke persoon in de werkelijkheid. Maar ook "Jan Janssen, geboren op 15 januari 2008" (de combinatie van de eigenschappen voornaam, achternaam en geboortedatum) kan een sleutelwaarde zijn voor een specifieke persoon in de werkelijkheid.

Voor een gegeven dat gaat over een letterlijk kenmerk, kunnen we direct de waarde van dat kenmerk zelf gebruiken, zoals in het gegeven uit bovenstaand voorbeeld: "10 februari 1970" is de waarde van het kenmerk waarover dit gegeven gaat. De waarde zelf kan letterlijk gebruikt worden in het gegeven, er is geen verschil tussen de manier waarop aan de waarde *refereren* en wat de waarde *betekent*. Dit gaat niet op voor rollen en categorische kenmerken.

Bij een rol is de invulling van de rol geen waarde, maar een (ander) domeinobject. Ook daar zullen we (dus) weer gebruik moeten maken van de identificerende kenmerken van dit domeinobject. Ook hier hebben we een sleutelwaarde nodig, zoals in het gegeven: "De persoon met BSN 123456782 is werknemer van het bedrijf met RSIN 287654321". Met dit gegeven wordt overigens hetzelfde bedoeld als met het gegeven "De persoon met de naam Jan is werknemer van het bedrijf met de naam Bakkerij Broodjes". Laatstgenoemde vorm is echter niet zo precies, aangezien het maar de vraag is of de eigenschap «naam» voldoende identificerend is.

Daarnaast zie je bij relaties en rollen dat je hetzelfde gegeven in drie verschillende vormen kunt uitdrukken, met elk dezelfde betekenis:

1. "De persoon met BSN 12345678 is werknemer van het bedrijf met RSIN 287
2. "De persoon met BSN 12345678 heeft als werkgever het bedrijf met RSIN
3. "Het bedrijf met RSIN 287654321 heeft als werknemer de persoon met BSN

Dat hier verschillende vormen mogelijk zijn, komt omdat de relatie zelf geen richting heeft, maar uitspraken over de relatie, de proposities, juist wel. En een gegeven is een vastgelegde uitspraak, dus kent *ook* een richting.

[!NOTE] Niet altijd is de sleutelwaarde beschikbaar die de voorkeur heeft op het moment dat we gegevens vastleggen. Als bijvoorbeeld het BSN van een persoon niet bekend is, maar we willen WEL gegevens vastleggen over een persoon, dan missen we de sleutelwaarde en kunnen we slechts gegevens OVER een domeinobject registreren, zonder dat we voldoende identificerende eigenschappen hebben. Zo wisten we in bovenstaand voorbeeld wel dat Jan een werknemer was van bakkerij Broodjes, maar verder wisten we niet zoveel over Jan. In zo'n geval zal de rolinvulling bestaan uit een beschrijving van het domeinobject (in dit geval Jan), zonder dat we expliciet verwijzen door middel van een sleutelwaarde.

Bij een indeling in categorieën is de invulling van een categorisch kenmerk geen letterlijke waarde, maar een benoemde categorie. Als we hierover gegevens willen vastleggen, dan volstaat het niet om te verwijzen naar de letterlijke waarde, maar hebben we een referentie nodig naar de categorie zelf. Het volstaat dan niet om de naam van de categorie te gebruiken als invulling voor dit kenmerk, immers: dan zou sprake zijn van een letterlijke waarde! We wensen daadwerkelijk te verwijzen naar de categorie zelf.

[!NOTE] De manier waarop verwezen kan worden naar de categorie zelf, is grotendeels een technische aangelegenheid. In Linked Data is het gebruikelijk om te verwijzen naar een URI, terwijl in relationele databases vaak gebruik wordt gemaakt van codes. Als bijvoorbeeld onderscheid gemaakt moet worden tussen de categorieën HOOG, MIDDEN en LAAG, dan zou in een Linked Data oplossing wellicht gekozen worden voor een URI <http://example.org/hoog> etc. In een relationele database zou gekozen kunnen worden voor de codes 0, 1 en 2. Van belang is dat bij categorieën daarbij deze codes niet moeten worden gezien als letterlijke waarden (de getallen 0, 1 en 2), maar als referenties naar de betreffende categorie.

§ 1.2.3.4 Wat voorbeelden

Nu we de begrippen rondom gegevens helder hebben, kunnen we een aantal voorbeelden van gegevens geven. We kunnen het zo over de volgende gegevens hebben:

- Er is een domeinobject met de voornaam "Jan" (de invulling voor de eigenschap «voornaam» van dit domeinobject is de waarde "Jan");
- Dit domeinobject heeft BSN 123456782 (de invulling voor de toegekende identificerende eigenschap «BSN» van dit domeinobject is de waarde "12345678");
- Dit domeinobject heeft als geboortedatum 25 mei 1970 (de invulling voor de eigenschap «geboortedatum» van dit domeinobject is de waarde "25 mei 1970");
- Dit domeinobject is een man (de invulling voor het categoriserend kenmerk «geslacht» is de categorie «mannelijk»);
- Dit domeinobject is de werknemer van een domeinobject met de naam "Bakkerij Broodjes" (de invulling voor de rol werkgever van [Jan] is [Bakkerij Broodjes]).
- Dit domeinobject is getrouwd met een domeinobject met de naam "Marie" (de invulling van de rol partner van [Jan] is [Marie]).

Deze zes uitspraken zijn zes gegevens die gegroepeerd kunnen worden tot één gegevensobject met als hoofdonderwerp het domeinobject met als sleutel de eigenschap «BSN» en met de sleutelwaarde "123456782".

§ 1.2.4 Typering van domeinobjecten

Nu we de concrete zaken hebben behandeld, kunnen we de stap maken naar de typering. Typering van domeinobjecten is zo de manier in MIM om aan te geven wat we wensen te identificeren en te onderscheiden, dwz: welke domeinobjecten dit zijn. Dit gebeurt door te beschrijven wat de over-

eenkomstige kenmerken zijn die domeinobjecten van een bepaald type gemeen hebben en hoe je deze domeinobjecten kunt identificeren met behulp van deze kenmerken.

[!CAUTION] In de tekst hieronder hebben we de terminologie voor typering zo consistent mogelijk doorgevoerd. Dit betekent daarmee ook dat we een aantal termen uit de huidige versie van MIM hebben aangepast, zonder direct een andere betekenis te beogen. Het gaat om de volgende termen:

- "attribuutsoort" naar "attribuuttype"
- "relatiesoort" naar "relatietype"
- "relatiesoortrol" naar "rolinvulling"
- "relatieklasse" naar "relatietype met kenmerken"

§ 1.2.4.1 Objecttype en populatie

Een OBJECTTYPE is een typering van gelijksoortige DOMEINOBJECTen

Een POPULATIE is de verzameling van alle mogelijke DOMEINOBJECTen die in de beschouwde werkelijkheid te onderscheiden zijn als OBJECTTYPE

Eerder hebben we een domeinobject beschreven als iets dat we willen onderscheiden in het domein. De manier om aan te geven dat we dat willen, is juist door het onderkennen van objecttypen. Immers: domeinobjecten bestaan in de werkelijkheid: die zullen we slechts beschrijven, nooit daadwerkelijk *zelf* opnemen in een model (!). Met het beschrijven van objecttypen geven we aan welke domeinobjecten we willen onderscheiden en identificeren. Dit is afhankelijk van de manier waarop we de werkelijkheid willen beschouwen: er is (dus) niet één unieke manier. Het is een keuze van de modelleur, gedreven door de kennis die er over dat domein nodig is.

Stel dat we het bijvoorbeeld in een domein willen hebben over auto's en fietsen. We hebben dan twee keuzes:

- We zijn geïnteresseerd in specifieke kenmerken om auto's en fietsen van elkaar te onderscheiden en te identificeren. In dat geval maken we de keuze om de objecttypen «Auto» en «Fiets» te onderkennen en afzonderlijk te beschrijven.
- We zijn niet geïnteresseerd in specifieke kenmerken om auto's en fietsen van elkaar te onderscheiden en te identificeren, maar we willen WEL kunnen aangeven dat een bepaald voertuig een auto of fiets is. In dat geval onderkennen we (slechts) het objecttype «Voertuig» met een

categorisch kenmerk «soort voertuig» met als mogelijke waarden de categorieën «auto» en «fiets».

[!NOTE] Een objecttype is een typering van gelijksoortige domeinobjecten. Je zou het daarmee ook kunnen hebben over een "domeinobjecttype". Het heeft echter onze voorkeur om de kortere term "objecttype" te hanteren. *Wat* je typeert is afhankelijk van je beschouwingsdomein. Waar alles een object zou kunnen zijn, beschouwen we iets pas een domeinobject, iets dat relevant is in ons beschouwingsdomein, als we hiervoor een objecttype modelleren in een conceptueel model. Anders gezegd: als een modelleur iets een «Objecttype» noemt, dan doet de modelleur daarmee **altijd** een uitspraak over het beschouwingsdomein, daarover kan en mag geen verwarring zijn.

§ 1.2.4.2 Categoriseren of typeren: objecttype vs categorie

De oplettende lezer zal zich afvragen wat het verschil is tussen een categorie en een objecttype. En dat is terecht. Want vaak kan iets zowel als een categorie en als een objecttype worden beschouwd. Je kunt het bijvoorbeeld hebben over de categorie «Homo Sapiens» als een categorie bij de indeling van levende wezens. Maar gelijktijdig kun je het hebben over het objecttype «Persoon» als typering van alle domeinobjecten die behoren tot die categorie. Het gaat om het doel waarvoor we typeren. Een categorie wordt gebruikt als onderdeel van een gegeven, terwijl een objecttype juist bedoeld is om een beschrijving te geven van de domeinobjecten waarin we zijn geïnteresseerd zijn, die we willen onderscheiden en waar we informatie over wensen.

- Met het introduceren van een objecttype beschrijf je dat je domeinobjecten wilt identificeren die van dit specifieke objecttype zijn en van deze domeinobjecten wil je afzonderlijke kenmerken en relaties onderkennen.
- Met het introduceren van een categorie beschrijf je dat je domeinobjecten wilt groeperen tot één categorie. Je wilt daarbij kunnen aangeven dat bepaalde domeinobjecten tot een specifieke categorie behoren, zonder daarbij daadwerkelijk domeinobjecten te willen zien als voorkomen van zo'n categorie.

Andersom wil je vaak juist informatie *over* een categorie weten. Bijvoorbeeld welke rechten- en plichten er van toepassing zijn op de categorie «BV». Zo kun je categorieën ook zien als conceptuele domeinobjecten. In dit voorbeeld zijn de verschillende categorieën te beschouwen als voorkomen van het objecttype «rechtsvorm».

Van een objecttype beschrijven we welke kenmerken we relevant vinden om te kunnen *weten* van objecten die tot een dergelijk objecttype behoren. Van een categorie beschrijven we juist wanneer een object tot een categorie *behoort*. Zo kent de BRT onder meer de objecttypen «Wegdeel», «Spoorbaandeel» en «Waterdeel». Elk van deze objecttypen heeft een categorisch kenmerk «fysiek

voorkomen». De volgende categorieën zijn daarbij onder meer mogelijk: «ondergronds», «overkluisd», «in tunnel», «op vast deel van brug». Niet elke categorie is mogelijk bij ieder objecttype. En hoewel het objecttype «Wegdeel in tunnel» opgenomen had kunnen worden in de BRT, is hiervoor niet gekozen.

Algemeen gesproken kun je stellen dat voor elk objecttype een overeenkomstige categorie denkbaar is, maar niet voor elke categorie zal een objecttype worden gemodelleerd:

- Bij elk objecttype horen domeinobjecten die voorkomen van dit objecttype. Vrijwel altijd zal relevant zijn om te kunnen weten van welk objecttype een specifiek domeinobject een voorkomen is. Om een dergelijk gegeven uit te kunnen drukken heb je (conform de definitie van een gegeven!) een kenmerk nodig. In dit geval gaat het om het categorisch kenmerk «type». Zoals in de zin: "[Jan] is een (voorkomen van het type) «persoon»".
- Objecttypen en categorieën kennen over het algemeen een hiërarchie. Zo zal een model van het dierenrijk een indeling bevatten waarin dieren worden getypeerd en gecategoriseerd conform de bekende biologische hiërarchie. In een dergelijk model zul je op een gegeven moment stoppen om nader te typeren, als over de "lagere" typen/categorieën geen extra informatie meer nodig is, behalve dan *dat* sprake is van een dergelijk categorie. Zo zou bijvoorbeeld (nog) onderscheid gemaakt kunnen worden in katten en honden (omdat we bijvoorbeeld van een kat andere kenmerken relevant vinden dan van honden), maar verder maken we geen afzonderlijke typering per ras. Wel zou het relevant kunnen zijn om een categorisch kenmerk «ras» op te nemen met de nadere onderverdeling.
- Er zijn meerdere indelingen in categorieën denkbaar voor een domeinobject. Zo wordt in de BRT onder meer onderscheid gemaakt in de objecttypen «wegdeel», «spoorbaan», «gebouw». Van een wegdeel is relevant wat de verhardingssoort is (zoals: asfalt, klinkers, gravel), maar ook het gebruik (zoals: fietspad, autoweg, voetpad). Je kunt langs verschillende facetten groeperingen maken van wegdelen. In de MIM standaard is er echter maar één subtypering mogelijk. Meerdere facetten zijn echter goed te ondersteunen door afzonderlijke categorische kenmerken te onderkennen.
- Categorieën kunnen tenslotte ook daadwerkelijk als voorkomens van een objecttype worden gemodelleerd, zodat aanvullende informatie over deze categorieën opgenomen kan worden in het model. Zo zou bijvoorbeeld van de categorie «Kat» en «Hond» vastgelegd kunnen worden dat het huisdieren betreffen, en van de categorie «Leeuw» vastgelegd kunnen worden dat dit juist geen huisdier is. Tegelijkertijd kan er een objecttype «Dier» zijn met kenmerken «haarkleur» en «diersoort». Een voorkomen van dit objecttype zou dan de haarkleur «zwart» kunnen hebben en de diersoort «kat». En omdat de categorie «kat» huisdieren betreft, weten we ook dat dit specifieke voorkomen een huisdier is.

[!NOTE] Dit onderscheid tussen typen en voorkomens verschilt per technische realisatie. Zo is in een relationele database een strikt onderscheid aanwezig tussen typen en voorkomens (respectievelijk tabellen en rijen) en ook in UML wordt dit onderscheid expliciet gemaakt (M0 versus M1). In RDF is dit onderscheid echter niet aanwezig en kunnen typen en voorkomens door elkaar gebruikt worden. Wel kan dit leiden tot beperkingen in de mogelijkheden voor inferencing (op basis van regels afleiden van nieuwe gegevens).

§ 1.2.4.3 Feiten, gegevens, informatiebehoefte en kennis

In MIM beschouwen we gegevens als vastgelegde uitdrukkingen over de feiten in de beschouwde werkelijkheid. Een feit is daarbij hetgeen waar is in die beschouwde werkelijkheid. Daarmee hoeft een gegeven nog niet geldig te zijn: een gegeven kan ook een verkeerde uitdrukking zijn van die feiten: de feiten kun je niet vastleggen, alleen uitspraken *over* de feiten, over de kenmerken van de domeinobjecten in de beschouwde werkelijkheid.

Een logisch gegevensmodel is een model van de (behoefte tot) uitwisseling, vastlegging en/of verwerking van deze gegevens. Een conceptueel informatiemodel is juist een model van de beschouwde werkelijkheid in termen van de typen domeinobjecten die we wensen te onderscheiden en de kenmerken waarover onze informatiebehoefte gaat. Een conceptueel informatiemodel beschouwt de werkelijkheid, voor zover we een informatiebehoefte over die werkelijkheid hebben. En die informatiebehoefte leidt (uiteindelijk) tot de verwerking van gegevens: een informatiebehoefte wordt vervuld door een gegevensverwerking.

In de voorbeelden hebben we het vaak over feiten zoals die zich daadwerkelijk voordoen. Zoals het feit: "[Jan] kwam op de savanne een dier tegen. Het betrof een «leeuw»". Of het feit: "[Jan] is getrouwd met [Marie]". Kennis gaat niet over dergelijke feiten. Kennis betreft juist de interpretatie, de betekenis die aan dergelijke feiten moet worden gegeven. Maar ook kennis kun je beschouwen als de werkelijkheid! Als we kennis beschouwen, dan beschouwen we juist feiten over die kennis als waar. Zo kan de kennis "Een «leeuw» is een gevaarlijk dier" ook als feit gezien worden! Door het toepassen van deze kennis, weten we dat Jan in gevaar is, hij doet er verstandig aan om naar een veilige plek te gaan.

Dergelijke kennisfeiten kun je ook uitdrukken in gegevens. In zo'n geval leg je kenmerken vast van de categorie «leeuw». Gegevens over categorieën zijn uitdrukkingen van kennis. Een model van de beschouwde werkelijkheid kan dus zowel een model zijn van de domeinobjecten die daadwerkelijk in die beschouwde werkelijkheid aanwezig zijn (zoals personen en dieren), maar een model van de beschouwde werkelijkheid kan ook betrekking hebben op de kennis daarover (zoals kennis over specieke categorieën van dieren).

Een ATTRIBUUTTYPE is een typering van een KENMERK, behorende tot een OBJECT-TYPE of RELATIETYPE

Waar een kenmerk (en daarmee ook een eigenschap) los kan staan van een domeinobject, geldt voor een attribuuotype dat deze getypeerd is in de context van een objecttype. Met andere woorden: het kenmerk «naam» kan zowel een kenmerk zijn van een schip als van een persoon, maar in geval van een attribuuotype heb je het dan over twee attribuuotypen: {Schip.naam} en {Persoon.naam}.

Waarden kunnen we daarbij ook typeren. Zo zal een kenmerk «geboortedatum» niet zomaar elke waarde kunnen zijn, maar kunnen we ook deze waarde typeren. Zo zal een geboortedatum van het waardetype «Datum» kunnen zijn.

Een WAARDETYPE is een typering van gelijksoortige WAARDEN

Een categorisch kenmerk heeft als invulling een categorie. Een attribuuotype van classificerende aard betreft de typering van dergelijke categorieën. Relevant daarbij is bovendien welke categorieën precies bij zo'n attribuuotype gebruikt kunnen worden. Dit is het classificatieschema. Dit kan een lijstje zijn van categorieën die zijn toegestaan, maar kan bijvoorbeeld ook een hiërarchie van categorieën omvatten.

[!NOTE] We spreken van "classificatieschema" en niet van "categorisatieschema", hoewel dat laatste wellicht meer voor de hand ligt omdat we het hebben over categorieën. Gekozen is voor de term "classificatieschema" omdat dit de algemeen gangbare term is. Daarnaast is een classificatieschema een formele ordening en dat is precies wat we bedoelen met dit schema: een formele ordening van categorieën, in een lijst of in een hiërarchie.

Een ATTRIBUUTTYPE VAN CLASSIFICERENDE AARD is een ATTRIBUUTTYPE als typering van een CATEGORISCH KENMERK

[!CAUTION] In de huidige versie van MIM geef je bij een attribuuotype op of deze classificierend is (ja/nee). Er is geen afzonderlijke term voor geïntroduceerd. Vandaar de term "attribuuotype van classificerende aard" in dit metamodel. Mogelijk is een andere term beter.

Een CLASSIFICATIESCHEMA is een systematische ordening van DOMEINOBJECTen in CATEGORIEën

Een eenvoudig voorbeeld van een classificatieschema is de lijst van primaire kleuren «rood», «geel», «blauw». Een voorbeeld van een hiërarchische classificatieschema is de [biologische inde-](#)

[ling van dieren](#) of de [indeling van boeken in een bibliotheek](#).

In de eerdere sectie kwam het onderscheid tussen een (eenvoudige) waarde en een complexe waarde naar voren. Ook dergelijke complexe waarden wil je typeren:

Een COMPLEX WAARDETYPE is een typering van gelijksoortige COMPLEXE WAARDEN

Een complex waardetype kan omschreven worden als een samenstel van de afzonderlijk benoemde waardetypen. Zo kan het complexe waardetype «Bedrag» omschreven worden als het samenstel van de waardetypen «Getal» en «Valuta». De waarde {getal: 25, valuta: «EURO»} zou een voorbeeld kunnen zijn van dit complexe waardetype. Merk op dat in MIM geen gebruik wordt gemaakt van volgorde. Dat sprake is van de twee waardetypen volgt uit de benoeming, niet uit de volgorde).

§ 1.2.4.5 Relatietypen en rollen

Een RELATIETYPE is een typering van gelijksoortige RELATIES

Een relatie is gelijksoortig als er sprake is van dezelfde rolinvullingen en als gelijksoortige proposities uitgedrukt kunnen worden met betrekking tot deze relaties. Daarom is onderdeel van de typering van de relatie ook de beschrijving van de rolinvullingen en de verwoording(en) waarmee de proposities kunnen worden uitgedrukt.

Een ROLINVULLING is een beschrijving van de invulling van een ROL in een RELATIETYPE door voorkomens van een OBJECTTYPE

Een VERWOORDING is een beschrijving van de manier waarop een voorkomen van een RELATIETYPE kan worden uitgedrukt in een propositie

Bij een verwoording kan gebruik worden gemaakt van de rollen die worden ingevuld bij het betreffende relatietype.

In een voorbeeld:

- De relatie tussen [Jan] en [Bakkerij broodjes] is te typeren als het relatietype met de naam "arbeidsrelatie"
- Het domeinobject [Jan] is een voorkomen van het objecttype «Persoon»
- Het domeinobject [Bakkerij broodjes] is een voorkomen van het objecttype «Organisatie»
- De rol «werkgever» wordt ingevuld door voorkomens van het objecttype «Organisatie» binnen het relatietype «Arbeidsrelatie» (organisaties zijn werkgevers bij een arbeidsrelatie)

- De rol «werknemer» wordt ingevuld door voorkomens van het objecttype «Persoon» binnen het relatietype «Arbeidsrelatie» (personen zijn werknemers bij een arbeidsrelatie)
- De relatie kan verwoord worden als: «werknemer» "is in dienst van" «werkgever»;
- De relatie kan (ook) verwoord worden als: «werknemer» "heeft" «werkgever»;
- De relatie kan (ook) verwoord worden als: «werkgever» "heeft" «werknemer».

[!NOTE] Een relatie kent altijd minimaal twee rolinvullingen. Het is daarbij niet verplicht dat een rolinvulling ook daadwerkelijk benoemd is, een naam heeft. Zo zou je een model kunnen maken waarin je beschrijft dat een persoon woont in een plaats, zonder te beschrijven hoe je de rolinvulling van de persoon noemt, dus:

- De relatie tussen [Jan] en [Amersfoort] is te typeren als een relatietype
- Het domeinobject [Amersfoort] is een voorkomen van het objecttype «Plaats»
- Het relatietype kan verwoord worden als: «Persoon» is woonachtig in «Plaats»
- De eerste rol wordt ingevuld door voorkomens van het objecttype «Persoon»
- De tweede rol wordt ingevuld door voorkomens van het objecttype «Plaats» In dit voorbeeld heeft de relatie niet echt een naam (maar wel een verwoording) en ook de rollen zijn niet benoemd.

§ 1.2.4.6 Relatietype met kenmerken

Uit de definitie van attribuuotype is (al) duidelijk dat een relatietype zelf ook kenmerken kan hebben. Zo kan uitdrukking worden gegeven aan het feit dat een relatie kenmerken kan hebben, zoals bijvoorbeeld de datum waarop een arbeidsrelatie is ontstaan.

§ 1.2.4.7 Conditities

De beschrijving van objecttypen bestaat niet alleen uit het toewijzen van kenmerken aan een objecttype. Het is ook relevant om te weten onder welke condities zo'n kenmerk geldt voor een domeinobject. En ook kunnen we beschrijven onder welke condities een domeinobject behoort tot de populatie van een objecttype.

Een CONDITIE is een noodzakelijke voorwaarde die moet gelden voor een typering

We onderscheiden verschillende soorten condities. De meest prominente specifieke condities zijn:

- **Cardinaliteit** is een conditie waarbij van een kenmerk wordt aangegeven hoeveel invullingen er voor één domeinobject minimaal en maximaal zijn.
- **Lengte** is een conditie waarbij van een kenmerk of waardetype wordt aangegeven hoe lang de invulling (de waarde) van dat kenmerk of waarden van een waardetype mag/mogen zijn.
- **Datatype** is een conditie waarbij van een kenmerk of waardetype wordt aangegeven wat voor datatype de invulling (de waarde) van dat kenmerk of waarden van een waardetype heeft/hebben. Een datatype is bijvoorbeeld: getal, tekst, datum, etc.

Daarnaast kent MIM twee condities waarmee het mogelijk is om aanvullende voorwaarden te beschrijven die niet met een specifieke conditie zijn uit te drukken:

- **Informele conditie** is een conditie die beschreven is in een natuurlijke taal, dwz: in een taal die mensen gebruiken in onderlinge communicatie.
- **Formele conditie** is een conditie die beschreven is in een machine-interpreteerbare taal.

[!NOTE] Wat hierbij opvalt is dat condities die gaan over letterlijke waarden (lengte, waardetype) zowel condities kunnen zijn voor gegevenstypen als voor kenmerken: het datatype van een gegevenstype over het kenmerk «geboortedatum» is exact hetzelfde datatype als voor dat kenmerk zelf. Wel kan een dergelijke conditie preciezer worden gemaakt als het gaat om gegevens, soms puur om praktische zin. Zo kan bij een kenmerk "voornaam" een waardetype «voornaamtypering» zijn opgenomen, waarin onder meer is opgenomen dat het gaat om een datatype tekst en dat een voornaam moet bestaan uit letters en geen cijfers mag bevatten. In de gegevensconditie kan daarbij aanvullend nog opgenomen worden dat de lengte maximaal 200 karakters is. Een waardetype kent daarmee een typering die betrekking heeft op het beschouwingsdomein, maar ook een stukje typering die kaders geeft voor het verwerkingsdomein.

§ 1.2.4.8 *Wat voorbeelden*

De begrippen die we hiermee gedefinieerd hebben, zijn behoorlijk abstract. Aangezien we het hier hebben over typering, was dat ook wel te verwachten. Daarom handig om een aantal voorbeelden te beschrijven: wat kunnen we nu met deze typering?

In het voorbeeld willen we de typering vastleggen uit het concrete voorbeeld hierboven over Jan.

- We beschrijven het objecttype met de naam "Persoon". Het domeinobject met de voornaam "Jan" typeren we als het objecttype «Persoon».

- Dit objecttype kent dus in ieder geval ook het attribuuttype met de naam "voornaam", dat het overeenkomstige kenmerk «voornaam» betreft.
- Aan het objecttype koppelen we ook het attribuuttype met de naam "BSN". Dit attribuuttype betreft het toegekend identificerend kenmerk «BSN».
- Ook koppelen we hier het attribuuttype met de naam "geboortedatum". Dit attribuuttype betreft het kenmerk «geboortedatum».
- Tenslotte koppelen we hier het attribuuttype met de naam "geslacht" aan. Dit attribuuttype betreft het categorisch kenmerk «geslacht».
- Als relatietype onderkennen we het relatietype met de naam "arbeidsrelatie". Dit relatietype relateert het objecttype «Persoon» aan het objecttype «Bedrijf».
- We onderkennen bij dit relatietype een rolinvulling met de naam "werkgever". Dit betreft de overeenkomstige rol «werkgever» die wordt ingevuld door (domeinobjecten getypeerd als) het objecttype «Bedrijf».
- We onderkennen bij dit relatietype een rolinvulling met de naam "werknemer". Dit betreft de overeenkomstige rol «werknemer» die wordt ingevuld door (domeinobjecten getypeerd als) het objecttype «Persoon».
- We onderkennen bij dit relatietype een verwoording: «werknemer» "werkt bij" «werkgever».

§ 1.2.4.9 Waardetypen vs categorieën en objecttypen

Als we een domein beschouwen, dan zullen daar bepaalde domeinobjecten in voorkomen waar we het "echt" over hebben. Een klantenadministratie zal het "echt" over klanten hebben, en de kenmerken die we van een klant belangrijk vinden, zoals bijvoorbeeld de naam, eerder gekochte artikelen en ook het land waar deze klant uit afkomstig is. België is daarmee (dus) ook een domeinobject in dit voorbeeld, net als de klant Jerome, afkomstig uit dit land en de artikelen die hij heeft eerder heeft gekocht. Het zal vermoedelijk echter niet nodig zijn om een afzonderlijke landenadministratie bij te houden. Dit is simpelweg een "lijstje" dat we eenmalig kunnen inladen.

Dergelijke lijstjes lijken bij de gegevensverwerking dan ook sterk op waardetypen waarbij een selectie uit een beperkt aantal categorieën c.q. waarden gekozen kan worden. Toch is er een belangrijk verschil bij het typeren:

- Een lijst van waarden bestaat uit (letterlijke) waarden: elke waarde *zelf* betekent niets meer dan de waarde zelf. Zo kun je het hebben over de lijst van getallen uit de fibonacci reeks kleiner dan 30, dwz: 1, 2, 3, 5, 8, 13 en 21. Dit is een (letterlijke) *waardelijst* en we hebben het hier over *letterlijke waarden*

- Een lijst van categorieën bestaat uit beschrijvingen van categorieën: de categorieën zijn aanduidingen voor een groep domeinobjecten die iets gemeen hebben. Zoals de categorieën «mannelijk» en «vrouwelijk». Je zou hiervoor de codes 1 en 2 kunnen gebruiken, maar daarmee zijn deze codes **geen** letterlijke waarden, maar nog steeds aanduidingen voor een groep domeinobjecten die iets gemeen hebben! Deze lijst hebben we een *classificatieschema* genoemd.
- Een lijst van domeinobjecten bestaat uit beschrijvingen van domeinobjecten, zoals de landen [België] en [Nederland]. Om te verwijzen naar het land [België] zou je daarbij de landcode 1 kunnen gebruiken, maar daarmee is deze code **geen** letterlijke waarde, maar de invulling van een identificerend kenmerk van het domeinobject België. Deze lijst is de *populatie* behorende bij het objecttype «Land».

§ 1.2.5 Typering van gegevens

§ 1.2.5.1 Gegevenstypen en gegevensobjecttypen

Na de typering van de onderwerpen uit het beschouwingsdomeinen, is de typering van de onderwerpen uit het verwerkingsdomein aan de beurt: gegevens en gegevensobjecten.

Een GEGEVENSTYPE is een typering van gelijksoortige GEGEVENS

Er is sprake van gelijksoortige gegevens, als deze gegevens over hetzelfde kenmerk van gelijksoortige domeinobjecten gaat. Dus bijvoorbeeld gegevenstypen die gaan over het kenmerk geboortedatum van een objecttype persoon. Er is sprake van een ander gegevenstype, als het kenmerk verschilt, of als het over een ander objecttype gaat.

Naast gegevens, kunnen we ook de gegevensobjecten typeren:

Een GEGEVENSOBJECTTYPE is een typering van gelijksoortige GEGEVENSOBJECTen

Er kan sprake zijn van gelijksoortige gegevensobjecten, als de gegevens die elk van deze gegevensobjecten bevat, van hetzelfde gegevenstype zijn. We kunnen met andere woorden gegevensobjecttypen typeren door te beschrijven welke gegevenstypen behoren tot dat gegevensobjecttype. Dit is echter niet voldoende. Gegevens gaan over domeinobjecten. Gegevensobjecten gaan daarmee ook over (diezelfde) domeinobjecten. Er is alleen sprake van gelijksoortige gegevensobjecten als de populatie waaruit deze domeinobjecten komen, dezelfde is. Een gegevensobjecttype {NederlandseInwoner} met als populatie de persoonsgegevens van alle Nederlanders, verschilt daarmee van een

gegevensobjecttype {EuropeseInwoner} met als populatie de persoonsgegevens van alle Europeanen.

Een gegevensobject kan gegevens omvatten die alleen gaan over één domeinobject, het hoofdonderwerp. Daarnaast onderkennen we ook de situatie dat een gegevensobject gaat over meerdere domeinobjecten, bijvoorbeeld een lijst met inwoners per provincie van Nederland. Maar ook voor dergelijke gegevensobjecten is sprake van dezelfde gelijksoortigheid bij de typering.

§ 1.2.5.2 *Identificeren van het hoofdonderwerp: sleutels*

Om het hoofdonderwerp van een gegevensobject te identificeren gebruiken we een sleutelwaarde. Zo is de waarde "123456782" van een BSN een unieke sleutelwaarde voor een specifieke persoon in de werkelijkheid. Een belangrijk aspect bij het typeren van gegevensobjecten is dan ook welke gegevenstypen bruikbaar zijn voor dergelijke sleutelwaarden. Zo wordt in dit voorbeeld het gegevenstype {Persoon.BSN} gebruikt. Dit gegevenstype is hier de *sleutel* voor het gegevensobjecttype Persoon.

Een SLEUTEL is een groep van één of meer GEGEVENSTYPEN waarmee een unieke aanduiding voor het HOOFDONDERWERP van een GEGEVENSOBJECT kan worden gevormd

Er kunnen meerdere (kandidaat) sleutels zijn voor een gegevensobjecttype. Een sleutel kan bestaan uit precies één gegevenstype, maar ook uit meerdere gegevenstypen. Een sleutel voor het gegevensobjecttype {Persoon} is bijvoorbeeld het gegevenstype {Persoon.BSN}, maar ook de groep van gegevenstypen {Persoon.voornaam, Persoon.achternaam, Persoon.geboortedatum, Persoon.geboorteplaats} zou een sleutel kunnen zijn voor dit gegevensobjecttype.

§ 1.2.5.3 *Varianten van gegevenstypen*

Vier varianten van gegevenstypen kunnen we onderscheiden:

1. Gegevenstypen waarbij sprake is van *letterlijke waarden*. Een gegevenstype over de eigenschap «leeftijd» of «voornaam» zijn voorbeelden van dergelijke gegevenstypen. Deze gegevenstypen zijn gerelateerd aan attribuuttypen.
2. Gegevenstypen waarbij sprake is van *categorieën*. Bij dit gegevenstype is geen sprake van een letterlijke waarde en refereert de waarde aan een betekenis die meer is dan de letterlijke waarde zelf. Een gegevenstype over het categorisch kenmerk «geslacht» is een voorbeeld van

een dergelijk gegevenstype. Deze gegevenstypen zijn gerelateerd aan attribuuttypen van classificerende aard.

3. Gegevenstypen waarbij sprake is van *complexe waarden*. Bij dit gegevenstype is sprake van een waarde die is opgebouwd uit enkele afzonderlijke onderdelen die gezamenlijk de complexe waarde vormen. Een gegevenstype over de eigenschap «lengte» is een voorbeeld van een dergelijke gegevenstype (in dit geval bestaat de complexe waarde uit een getal en een eenheid). Deze gegevenstypen zijn gerelateerd aan attribuuttypen met complexe waardetypen.
4. Gegevenstypen waarbij sprake is van een *sleutelwaarde die refereert aan een ander domeinobject*. Bij dit gegevenstype is sprake van een sleutelwaarde die een referentie betreft naar dit andere domeinobject. Deze gegevenstypen zijn gerelateerd aan verwoordingen van relaties en rolinvullingen.

Merk op: er bestaat op het niveau van gegevens (dus) slechts gegevensobjecttypen en gegevenstypen. Er bestaat niet zoiets als een gegevensrelatietype. En hoewel je in een plaatje een lijntje kunt tekenen die "lijkt" op een relatietype, betreft dit lijntje niets meer of minder dan een gegevenstype van variant (4). Een dergelijk lijntje **MOET** dan ook altijd gericht zijn: vertrekken vanuit het gegevensobjecttype waar het betreffende gegevenstype toe behoort.

Merk op: ook een eigenschap «BSN» betreft een gegevenstype waarbij sprake is van een letterlijke waarde. Hoewel een BSN ook gebruikt zou kunnen worden als een sleutelwaarde die refereert aan een ander domeinobject (bijvoorbeeld in een gegeven over een kenmerk «(heeft) ouder»), wordt in dit geval de eigenschap niet gebruikt als referentie, maar als (toegekende) eigenschap aan het onderwerp van het betreffende gegeven. Zo is in het gegeven "[Jan] heeft BSN 123456782" de waarde van de eigenschap een letterlijke waarde (namelijk: "123456782") en wordt hiermee niet een verwijzing naar Jan zelf bedoeld.

§ 1.2.5.4 Gegevens over relaties

In een logisch gegevensmodel bestaan slechts gegevensobjecttypen. Terwijl in een conceptueel informatiemodel zowel objecttypen als relatiotypen bestaan. Je kunt je dan ook de vraag stellen hoe we omgaan met gegevens over relaties: hoe leg je die vast?

[!NOTE] Als je een grafische weergave maakt van een logisch gegevensmodel, dan zullen daar vanzelfsprekend ook pijlen staan *tussen* gegevensobjecttypen. Deze pijlen representeren echter niet een relatietype zoals in een conceptueel informatiemodel, maar representeren een gegevenstype-4: een gegevenstype waarbij sprake is van een sleutelwaarde die refereert aan een ander domeinobject. De pijl geeft daarmee de richting aan: de pijl begint bij het onderwerp van dit gegevenstype, en eindigt bij de referent.

Daarvoor heb je als gegevensmodelleur vier keuzes. Welke keuze je kiest, hangt sterk af van de manier waarop je de gegevens wilt verwerken. Stel we beschouwen het relatietype «arbeidsrelatie» uit de voorgaande secties. We hebben dan de volgende mogelijkheden in het gegevensmodel:

1. We leggen de gegevens over de arbeidsrelatie alleen vast bij het gegevensobjecttype {Persoon}, een gegevensobjecttype met als hoofdonderwerp voorkomens van het objecttype «Persoon». Dit zou kunnen leiden tot een gegevenstype over het relationele kenmerk «werkgever», zoals in het gegeven "[Jan] heeft «werkgever» [Bakkerij Broodjes]".
2. We leggen de gegevens over de arbeidsrelatie alleen vast bij het gegevensobjecttype {Organisatie}, een gegevensobjecttype met als hoofdonderwerp voorkomens van het objecttype «Organisatie». Dit zou kunnen leiden tot een gegevenstype over het relationele kenmerk «werknemer», zoals in het gegeven "[Bakkerij Broodjes] heeft «werknemer» [Jan]".
3. We leggen de gegevens over de arbeidsrelatie bij zowel het gegevensobjecttype {Persoon} als {Organisatie} vast. In dit geval is sprake van redundantie: de twee gegevens gaan over hetzelfde feit uit de werkelijkheid. Bij de verwerking van de gegevens is extra aandacht nodig om te voorkomen dat dit leidt tot onduidelijkheid over de geldigheid van deze twee gegevens.
4. We introduceren een nieuw gegevensobjecttype {Arbeidsrelatie} dat de gegevens omvat met betrekking tot de relatie tussen [Jan] en [Bakkerij Broodjes]. Dit zou kunnen leiden tot twee gegevenstypen met betrekking tot de kenmerken van de arbeidsrelatie, zoals in de twee gegevens: "[Arbeidsrelatie tussen [Jan] en [Bakkerij Broodjes]] betreft «werknemer» [Jan]" en "[Arbeidsrelatie tussen [Jan] en [Bakkerij Broodjes]] betreft «werkgever» [Bakkerij Broodjes]".

Merk op dat als een relatietype eigen kenmerken heeft (zoals bijvoorbeeld de begindatum van de arbeidsrelatie) alleen keuze (4) nog overblijft.

§ 1.2.5.5 Speciaal soort gegevensobjecttypen

Een gegevensobjecttype kan sterk lijken op een objecttype. We noemen dat *isomorf*. In dat geval gaan de gegevens die bij één gegevensobject worden bijgehouden over eigenschappen van één domeinobject van dat objecttype. Maar het is ook denkbaar om gegevens over andere objecten bij een gegevensobject te plaatsen.

Als een voorkomen van een gegevensobjecttype precies één hoofdonderwerp heeft en de sleutel is bekend, dan is sprake van een *eenduidig gegevensobjecttype*. Je weet dan van een gegevensobject over welk domeinobject dit gegevensobject in hoofdzaak gaat. Zo is bij voorkomens van het gegevensobjecttype {Ingezetene} met als sleutel {Ingezetene.BSN} duidelijk dat dit gegevensobjecttype isomorf is met het objecttype «Ingezetene» en dat gegevensobjecten hiervan 1-op-1 overeen-

komen met de afzonderlijke domeinobjecten: precies één gegevensobject gaat over precies één domeinobject.

Als daarbij ook nog geldt dat de gegevens in dit gegevensobject allemaal over dit ene hoofdonderwerp gaan, dan is sprake van een *strikt eenduidig gegevensobjecttype*. Mocht dit niet het geval zijn, dan is het nodig om een pad te beschrijven voor de gegevenstypen waarvoor dit niet geldt. Als bijvoorbeeld het gegevenstype "woonplaatsnaam" is opgenomen bij een gegevensobjecttype {Persoon}, dan is nodig om te weten hoe je van een persoon komt bij de naam van de woonplaats. Het pad zou kunnen zijn: {Persoon.woonplaatsnaam} = {Persoon} -> woont in -> {Plaats.naam}.

Bij een beschrijvend gegevensobjecttype is er juist geen (goede) sleutel beschikbaar. Een voorbeeld is het gegevensobjecttype {persoonssignalement}. Hoewel dit gegevensobjecttype over precies één hoofdonderwerp gaat (een persoon), is er geen duidelijkheid over welk domeinobject zo'n signalement precies gaat. We weten echter wel bepaalde gegevens over dit domeinobject (bijvoorbeeld: lengte, haarkleur, brildragend, etc). Een ander voorbeeld is het gegevensobjecttype «contactpersoon». Ook hier gaat het om precies één hoofdonderwerp en ook hier is er geen duidelijkheid over welk domeinobject het precies gaat. Wellicht weten we alleen een voornaam en een telefoonnummer. Voor dergelijke gegevensobjecttypen is een administratieve sleutel nodig die niet verwijst naar het domeinobject, maar juist naar het gegevensobject, de beschrijving *zelf*. Technisch gezien kan dit ook opgelost worden door de beschrijving onderdeel te maken van een ander gegevensobjecttype (technisch gezien vergelijkbaar met een complexe waarde, maar logisch gezien net wat anders). Bij een signalement kan dit bijvoorbeeld het gegevensobjecttype «aangifte» zijn, en bij een contactpersoon bijvoorbeeld het gegevensobjecttype «organisatie».

[!NOTE] In een logisch gegevensmodel kan de relatie tussen een beschrijvend gegevensobjecttype en het gegevensobjecttype waarbij de gegevens worden geplaatst afgebeeld worden met een composition-relatie. Deze relatie bestaat echter *alleen* op het logisch niveau! Anders dan bijvoorbeeld bij een vliegtuigmotor en een vliegtuig is een persoon immers geen feitelijk onderdeel van een aangifte of organisatie!

§ 1.2.5.6 Speciaal soort gegevenstypen

Een gegevenstype betreft over het algemeen gegevens over precies één kenmerk van precies één object. Het is ook denkbaar om gegevens vast te leggen die betrekking hebben op een samenstelling van meerdere kenmerken of van meerdere objecten.

- Bij een *direct gegevenstype* is sprake van gegevens over precies één kenmerk van precies één domeinobject. Bijvoorbeeld het gegevenstype {Persoon.lengte}. Dit gegevenstype gaat over het kenmerk «lengte» van een «Persoon»;

- Bij een *indirect gegevenstype* is sprake van gegevens die gaan over een kenmerk van een ander domeinobject dan het hoofdonderwerp. Bijvoorbeeld het gegevenstype {Persoon.woonplaatsnaam}. Dit gegevenstype gaat over het kenmerk «naam» van een «Plaats». Je kunt niet direct van het hoofdonderwerp bij dit kenmerk uitkomen, dit gaat indirect via de relatie tussen het hoofdonderwerp en zijn woonplaats: {Persoon.woonplaatsnaam} = {Persoon} -> woont in -> {Plaats.naam}.
- Bij een *samengesteld gegevenstype* is sprake van gegevens die worden gecombineerd tot één gegeven. Bijvoorbeeld een totaalbedrag op een factuur, of de volledige naam van een persoon als samenstelling van de voor- en achternaam.

§ 1.2.5.7 Gegevensverzamelingen en referentielijsten

Bij de typering van domeinobjecten (zie paragraaf 'Waardetypen vs categorieën en objecttypen') kwam het onderscheid tussen waardelijsten, classificatieschemas en populaties aan bod. Daarbij werd duidelijk dat:

- Een *waardelijst* een lijst van *waarden* is;
- Een *classificatieschema* een ordening beschrijft in *categorieën*;
- Een *populatie* een opsomming van *domeinobjecten* betreft.

Bij de typering van gegevensobjecten onderkennen we naast deze drie nog een classificatielijst. Waar een classificatieschema diepgang kent, het kent een boomstructuur van categorieën (bijvoorbeeld de indeling van biologische soorten van flora en fauna), geldt dat voor een classificatielijst niet. Een classificatielijst zou je kunnen zien als een classificatieschema zonder diepgang: alle categorieën in één lijst. Een classificatielijst zou bijvoorbeeld de lijst kunnen zijn van alle categorieën op het niveau van biologische soort.

Het onderscheid tussen waardelijsten, classificatielijsten en populaties is vooral een conceptueel onderscheid. Puur als lijstje van gegevens is het onderscheid hiertussen veel minder duidelijk. Hieronder drie voorbeelden:

A. De lijst getallen uit de Fibonacci-reeks kleiner dan 30. Deze lijst bestaat uit de getallen 1, 2, 3, 5, 8, 13 en 21. B. De lijst van olympische sporten die behoren tot de meerkamp van de mannen. Deze lijst bestaat onder meer uit de sporten «100 meter sprint», «verspringen» en «hoogspringen». C. De lijst van landen die een ISO 3166-1 landencode hebben. Deze lijst bestaat onder meer uit de landen [België], [Frankrijk], [Egypte], [Thailand] en [Argentinië].

Dergelijke lijsten worden vaak gebruikt als referentielijsten. Het gaat dan om lijsten die gebruikt worden om naar te refereren, zonder dat de inhoud van die lijsten wordt bijgehouden in de betreffende administratie of onderdeel is van een gegevensuitwisseling. Bij dit verwijzen maakt het niet

zo heel veel uit of er nu sprake is van een waardelijst, classificatielijst of populatie. Als verwijzing wordt een waarde gebruikt. Dat kan de letterlijke waarde zelf zijn, de aanduiding van de categorie of de identificatie van het domeinobject, in bovenstaande voorbeelden respectievelijk "1", "verspringen" en "België".

Toch zijn er wel belangrijke verschillen, waarbij het goed is om deze drie lijsten uit elkaar te houden. Met name omdat bij de beschrijving van deze lijsten zal verschillen, afhankelijk van wat voor soort het lijst het betreft:

- Lijst A betreft een *waardelijst*. De lijst bestaat uit waarden. Dergelijke waarden kennen geen andere kenmerken dan de waarde zelf (het is niet zinvol om te spreken over "code" of "label" of iets dergelijks, de waarde betekent immers -per definitie- niets anders dan de waarde zelf);
- Lijst B betreft een *classificatielijst*. De lijst bevat categorieën. Met «verspringen» wordt de categorie bedoeld. Het is zelfs niet zomaar verspringen, maar moet voldoen aan een veelheid aan regels voordat je kunt spreken van verspringen als olympische sport. Je zou van zo'n categorie informatie kunnen vastleggen zodat duidelijk wordt wat er precies met zo'n categorie wordt bedoeld. Bovendien kan ook een ander kenmerk gebruikt worden voor de referentie naar zo'n categorie. Bijvoorbeeld een URI of een code.;
- Lijst C betreft een *populatie*. We hebben het hier over domeinobjecten van het objecttype «Land». En daarbij een specifieke populatie (dus niet alle mogelijke voorkomens van dit objecttype), namelijk die domeinobjecten met de eigenschap ISO landencode. De modellering van de gegevens over deze populatie zijn feitelijk niet heel anders dan die van elk ander objecttype. En ook hier geldt dat je een ander kenmerk zou kunnen gebruiken voor de referentie naar een voorkomen van dit domeinobject. Bijvoorbeeld een URI of een code (wat bij de ISO-3166-1 landencodelijst wel voor de hand ligt!)

§ 1.2.5.8 Gegevenstypen en gegevens m.b.t. waardelijsten

Een gegevenstype dat gegevens typeert over kenmerken, kun je koppelen aan een waardelijst: dit zijn dan de waarden die je mag gebruiken. Zo kun je een gegevenstype specificeren van het kenmerk "storypoints" met als waardelijst die hierboven genoemde waardelijst. Deze waardelijst bestaat uit de volgende gegevens:

Waardelijstnaam	Waarde
Fibonachegetallen onder 30	1
Fibonachegetallen onder 30	2
Fibonachegetallen onder 30	3
Fibonachegetallen onder 30	5

Waardelijstnaam	Waarde
Fibonachegetallen onder 30	8
Fibonachegetallen onder 30	13
Fibonachegetallen onder 30	21

§ 1.2.5.9 Gegevenstypen en gegevens m.b.t. classificatielijsten

Een gegevenstype dat gegevens typeert over categorieën, kun je koppelen aan een referentielijst. Deze referentielijst bevat referentiewaarden. Anders dan letterlijke waarden, verwijzen deze referentiewaarden naar afzonderlijke categorieën. Elke referentiewaarde is een sleutelwaarde van de betreffende categorie. Zo kun je een gegevenstype specificeren van de eigenschap "onderdeel" met als referentielijst een lijst met referentiewaarden die refereren aan de hierboven genoemde categorieën. Relevante eigenschappen van deze categorieën kunnen daarbij bijvoorbeeld zijn: code, naam en betekenis. De code is de identificerende eigenschap van de categorie die gebruikt wordt als de referentiewaarde in het gegeven. De naam is de identificerende eigenschap zoals deze voor mensen herkenbaar is en de betekenis is de eigenschap die verwijst naar een begrip waarin de betekenis wordt uitgelegd. Deze referentielijst bestaat uit de volgende gegevens:

Waardelijstnaam	Code	Naam	Betekenis
Mannen meerkamponderdelen	VER	Verspringen	Long jump
Mannen meerkamponderdelen	100V	100 meter sprint vlak	100 metres
Mannen meerkamponderdelen	KOGEL	Kogelstoten	Shot put
Mannen meerkamponderdelen	HOOG	Hoogspringen	High jump
Mannen meerkamponderdelen	400V	400 meter vlak	400 metres
Mannen meerkamponderdelen	110H	110 meter horden	100 metres hurdles
Mannen meerkamponderdelen	DISCUS	Discuswerpen	Discus throw
Mannen meerkamponderdelen	POLSTOK	Polstokhoogspringen	Pole vault
Mannen meerkamponderdelen	SPEER	Speerwerpen	Javelin throw
Mannen meerkamponderdelen	1500V	1500 meter vlak	1500 metres

Merk op dat deze classificatielijst eenvoudig is te verdiepen tot een classificatieschema door een extra eigenschap als "categorie" op te nemen, waarbij elke sportsoort weer verwijst naar algemenere categorieën, bijvoorbeeld:

Code	Categorie	Naam	Betekenis
VER	SPRING	..	..
100V	LOOP	..	..
KOGEL	WERP	..	..
SPRING	ATHL	Springonderdeel	..
LOOP	ATHL	Looponderdeel	..
WERP	ATHL	Werponderdeel	..
ATHL	SPORT	Athletiek	..
SPORT	Sport	..	..

§ 1.2.5.10 Gegevenstypen en gegevens m.b.t. populaties

Een gegevenstype dat gegevens typeert over populaties, kun je ook koppelen aan een referentielijst. Deze referentielijst bevat referentiewaarden. Anders dan letterlijke waarden, verwijzen deze referentiewaarden naar afzonderlijke domeinobjecten uit deze populatie. Deze referentiewaarden zijn feitelijk de sleutelwaarden van het betreffende domeinobject. Zo kun je een gegevenstype specificeren van de rol "land van herkomst" met als referentielijst een lijst met referentiewaarden die refereren aan de hierboven genoemde landen met een ISO 3166-1 landencode. Relevante eigenschappen van deze landen kunnen daarbij bijvoorbeeld zijn: naam, alpha-2 en alpha-3. Daarbij is zowel de alpha-2 als alpha-3 eigenschap bruikbaar als referentiewaarde. De naam is de identificerende eigenschap zoals deze voor mensen herkenbaar is (in dit geval degenen die de Nederlandse taal machtig zijn). Deze referentielijst bestaat uit de volgende gegevens (alleen de gegevens van de eerste drie landen zijn hieronder getoond):

Waardelijstnaam	Alpha-2	Alpha-3	Naam
ISO 3166-1	AF	AFG	Afghanistan
ISO 3166-1	AX	ALA	Åland
ISO 3166-1	AL	ALB	Albanië

§2. Conformiteit

Naast onderdelen die als niet-normatief gemarkeerd zijn, zijn ook alle diagrammen, voorbeelden, en noten in dit document niet-normatief. Verder is alles in dit document normatief.

Het trefwoord *MOET* in dit document moet worden geïnterpreteerd als in [BCP 14 \[RFC2119\]](#) [\[RFC8174\]](#) als, en alleen als deze in hoofdletters zijn weergegeven, zoals hier getoond.

§ Index

§ Termen gedefinieerd door deze specificatie

§ Termen gedefinieerd door verwijzing

§A. Referenties

§A.1 Normatieve referenties

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997.
Best Current Practice. URL: <https://www.rfc-editor.org/info/rfc2119/>

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017.
Best Current Practice. URL: <https://www.rfc-editor.org/info/rfc8174/>